IntroSat.Platform: краткое руководство

IntroSat.Platform – обучающий конструктор спутника форм-фактора CubeSat 3U, предназначенный для проектирования собственной космической миссии на его борту с возможностью последующих испытаний в форме стратосферного запуска на высоту до 25 километров. Конструктор содержит в себе все необходимые служебные системы и основные конструкционные элементы спутника, позволяя легко интегрировать в него полезную нагрузку, разработанную учащимися.или

Вместо эпиграфа…

1. При оценивании, сколько времени понадобится на выполнение задачи, смело умножай полученное на три.
2. Datasheet и Application Note на устройство могут сказать о нём больше, чем любой гайд (и даже этот).

То, что вы точно захотите найти в этом файле:

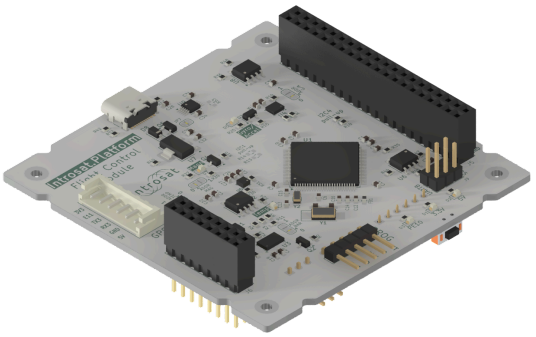
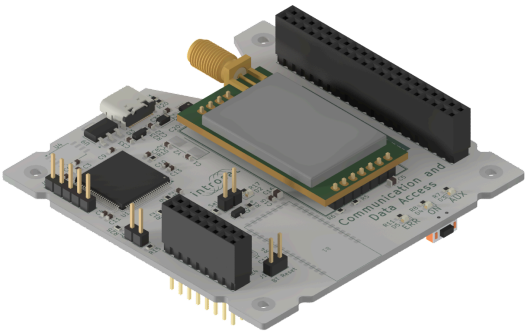
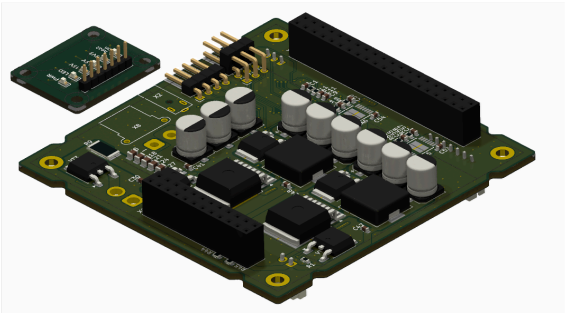
- Установка ПО и библиотек
- Распиновка шины IntroBus_L для подключения полезной нагрузки
- Особенности схемы питания конструктора

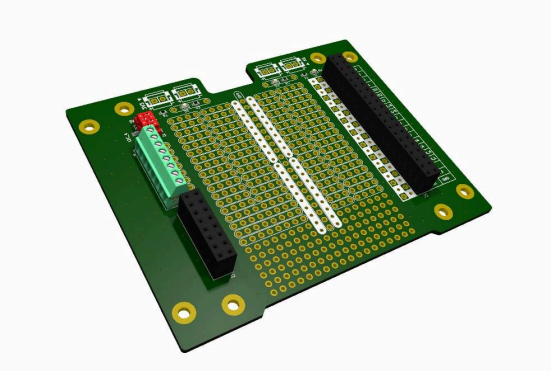
Состав конструктора IntroSat.Platform

A kit specification

Модули служебных систем

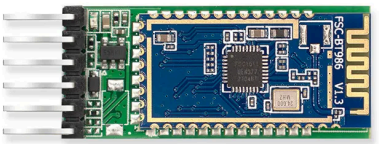
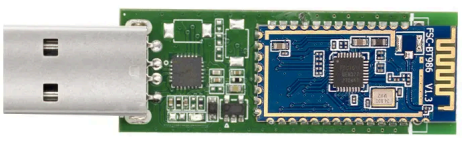
Служебные системы конструктора представляют собой печатные платы, соединяемые в стек посредством двухрядных штырьковых сквозных разъёмов.

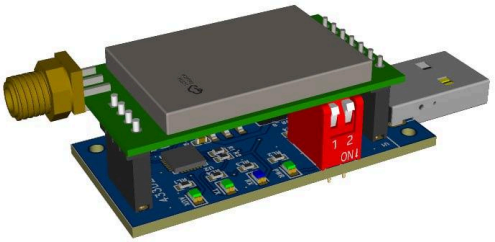
Модуль полётного контроллера	
Плата связи	
Модуль электропитания	

Интерфейсный модуль	

Вспомогательные модули и устройства

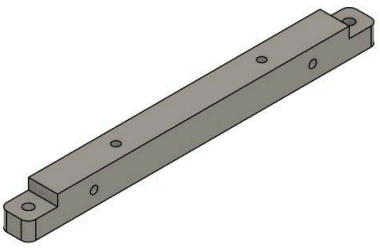
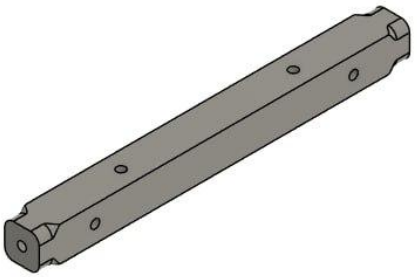
При работе с конструктором также используются следующие вспомогательные модули и устройства:

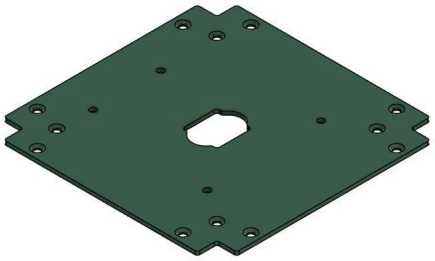
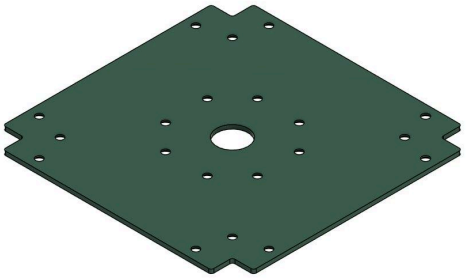
Программатор ST-Link	
USB-UART конвертер	
Bluetooth модуль	
Bluetooth модуль со встроенным USB-UART конвертером	

Модуль LoRa-UART	
Источник питания 12 В	
Переходник для источника питания на XT30	

MicroSD card	

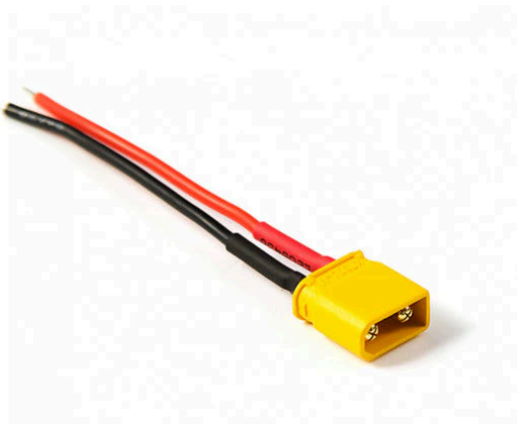
Конструкционные элементы

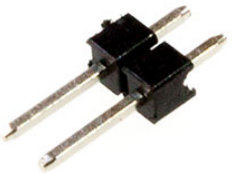
Rails 3U	
Cutout bars	
Holed bars	

Top panel	
Bottom panel	

Коммутация и крепёж

M2.5 round head screws (different length)	
M2.5 flat head screws (different length)	
M2.5 nuts	

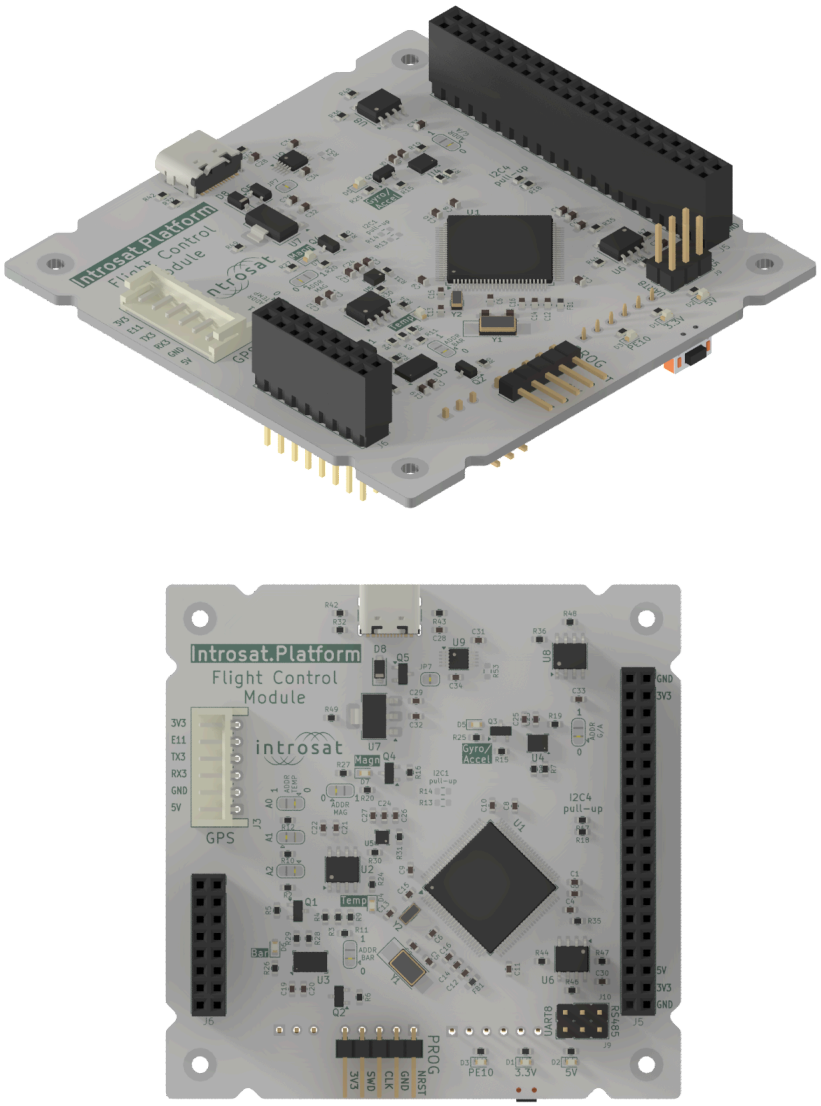
M2.5 brass standoffs (different length)	
M2.5x15 brass standoffs (specifically for service boards securing)	
Solderless jumper wires (Male to Male, Female to Female, Male to Female)	
USB A to USB Type C cable	
XT30 Male cable	

JST “Female to Female” cable	
JST B2B-EH-A (Male)	
PLS-2	
Terminal block, 2 pins, 2.54 mm	
Antenna 433 MHz	

Pigtail SMA-Male right angle to SMA-Female	

Модуль полётного контроллера (Flight Controller Module)

Общая информация



Внешний вид платы модуля полётного контроллера

Плата управления полётом является основной для всей системы. На ней расположен микроконтроллер STM32H750VBT6. Именно он отвечает за все действия, выполняемые аппаратом, и взаимодействие с полезной нагрузкой.

Габариты платы	76x76 мм
Расстояние между отверстиями для	65 мм

крепления	
Модель микроконтроллера	STM32H750VBT6
Напряжение питания микроконтроллера и датчиков	3,3 В
Напряжение питания драйверов интерфейсов RS485 и CAN, Bluetooth-модуля	5 В (номинальное)
Подключение источников питания	Соединение в стек с платой модуля электропитания; подключение к порту компьютера / адаптеру через кабель USB A - USB Type-C; питание микроконтроллера через программатор
Служебные датчики	LSM6DS3, LIS3MDL, MS5611, LM75A
Система энергонезависимого хранения информации	MicroSD карта
Драйвер интерфейса RS485	ADM3485EARZ
Драйвер интерфейса CAN	MAX3051
Встроенный USB-UART конвертер	CH340X, используется для обмена данными при подключении через порт USB Type-C
Беспроводная связь	Подключение Bluetooth-модуля в гнездо на плате; соединение в стек с платой связи для обмена данными с помощью радиомодуля LoRa
Интерфейс выделенного порта для программирования и отладки	SWD (Serial Wire Debug)
Подключение полезной нагрузки	На этапе макетирования и тестирования: с помощью проводов для беспаечной сборки к разъёму IntroBus_L; при финальной сборке: полезная нагрузка коммутируется через плату интерфейсного модуля,

	с которым модуль полётного контроллера соединяется в стек

Кроме контроллера на этой плате расположены основные датчики, необходимые для получения телеметрии в полёте:

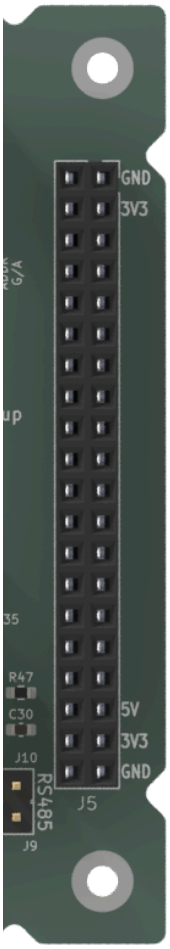
- LSM6DS3 выполняет две функции:
    - акселерометра, для определения ускорения аппарата
    - гироскопа, для определения угловой скорости аппарата
- LIS3MDL – магнитометр, для определения индукции магнитного поля
- MS5611 – барометр, для определения давления и вычисления приблизительной высоты
- LM75A – датчик температуры, для определения температуры платы

Шина IntroBus

Для соединения плат служебных модулей в стек используется шина конструктора под названием IntroBus. Она состоит из двух двухрядных штырьковых сквозных разъёмов: IntroBus_S (шестнадцати контактный) и IntroBus_L (сорокаконтактный).

Разъём IntroBus_S является служебным (разъём J6 на плате модуля полётного контроллера). Подключение к нему строго запрещено.

Разъём IntroBus_L (J5) служит для подключения устройств полезной нагрузки к микроконтроллеру.



Разъём IntroBus_L (J6) на модуле полётного контроллера

Шелкографией на плате подписаны (для обоих рядов разъёма) контакты питания полезной нагрузки: 3.3 и 5 В. Питание подаётся на эти контакты только в случае соединения платы в стек с модулем электропитания и включения на модуле электропитания каналов 3.3 и 5 В посредством установки соответствующих перемычек. Без модуля электропитания на контактах питания полезной нагрузки напряжения не будет.

Через разъём IntroBus_L обеспечивается подключение устройств полезной нагрузки к микроконтроллеру через разнообразные интерфейсы; также разъём позволяет использовать такую встроенную периферию микроконтроллера, как:

- аналогово-цифровой преобразователь (ADC)
- цифро-аналоговый преобразователь (DAC)
- компаратор (COMP)

- операционный усилитель (OP_AMP)
- цифровой фильтр для сигма-дельта модулятора (DFSDM)
- таймер высокого разрешения (HRTIM).

Доступные для коммуникации с полезной нагрузкой интерфейсы:

- I2C
- SPI
- UART – 3 шт.
- RS485 (в таком случае становится недоступен UART8)
- CAN

Любой пин микроконтроллера также может быть использован как интерфейс GPIO.

Распиновка разъёма IntroBus_L представлена в таблице ниже. Многие пины имеют несколько альтернативных функций, не приведённых в таблице. Более подробную информацию об альтернативных функциях пинов можно найти в datasheet на микроконтроллер STM32H750VBT6.

IntroBus_L_2x20 (FCM)			
GND	GND	GND	**GND**
+3V3_IS	3V3	3V3	+3V3_IS
OPAMP_OUT	PE7	PB0	ADC
OPAMP_INM	PE8	PB1	ADC
OPAMP_INP	PE9	PB15	MOSI2
ADC / DFSDM_CKIN	PC0	PB14	MISO2
ADC / DFSDM_DATIN	PC1	PB10	SCK2
ADC / DFSDM_CKOUT	PC2_C	PE0/A	RX8/A
ADC	PC3_C	PE1/B	TX8/B
COMP_INM	PC4	PD9	RX3
COMP_OUT	PC5	PD8	TX3
HRTIM_CHA1	PC6	PB12	RX5
HRTIM_CHA2	PC7	PB13	TX5
COMP_INP	PB2	PD13	SDA4
DAC_EXT	PB9	PD12	SCL4
DAC_OUT1	PA4	CAN_H	CAN_H
DAC_OUT2	PA5	CAN_L	CAN_L
+5V_IS	5V	5V	+5V_IS
+3V3_IS	3V3	3V3	+3V3_IS
GND	GND	GND	**GND**
Вид на плату сверху			

Даже на этапе макетирования и тестирования для подключения полезной нагрузки к микроконтроллеру удобнее воспользоваться платой интерфейсного модуля, так как все используемые контакты шины IntroBus подписаны шелкографией рядом с разъёмом.

Подключение датчиков

Подключение всех служебных датчиков, расположенных на плате полётного контроллера, осуществляется через шину I2C1. Она является служебной и служит только для коммуникации внутри блока служебных систем; подключить к ней полезную нагрузку нельзя.

Шина I2C1 использует следующие пины микроконтроллера:

SDA1	PB7

SCL1	PB6

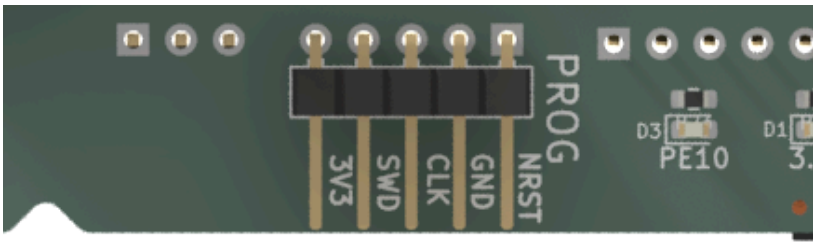
Также несколько пинов микроконтроллера соединены с некоторыми датчиками для получения дополнительных сигналов от них:

Вывод датчика	**Пин микроконтроллера**	**Назначение**
LM75A: вывод OS	PE2	Прерывание
LSM6DS3: вывод INT1	PA15	Прерывание
LIS3MDL: вывод INT	PB3	Прерывание
LIS3MDL: вывод DRDY	PB4	Готовность данных

Более подробную информацию об использовании этих дополнительных выводов можно найти в Datasheet и Application Note на датчики.

Разъём для прошивки и отладки

Основным способом загрузки прошивки на микроконтроллера служит угловой разъём PLS-5R. На плате он подписан как PROG.



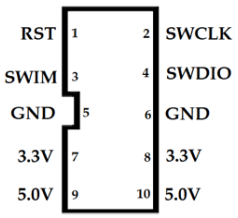
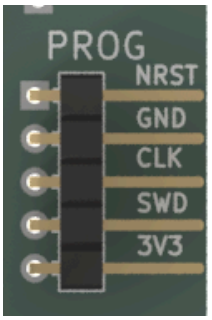
Разъём для прошивки микроконтроллера

Для перепрошивки микроконтроллера к данному разъёму требуется подключить программатор ST-Link, используя 4 провода “мама-мама”.



Программатор ST-Link

Загрузка прошивки происходит через интерфейс SWD (Serial Wire Debug). Также, в случае, если среда разработки обладает таким функционалом, этот интерфейс используется для отладки – работы с микроконтроллером в режиме построчного выполнения программы с возможностью посмотреть текущие значения переменных и содержимое регистров. Подключение осуществляется согласно таблице:

ST-Link	FCM
 	
SWCLK >--- ---> CLK SWDIO >--- ---> SWD GND >--- ---> GND	

Если к плате полётного контроллера не подключен иной источник питания, можно соединить контакты 3.3V программатора ST-Link с контактом 3V3 на разъёме PROG платы. Таким образом служебные системы платы (и микроконтроллер) будут обеспечены питанием 3.3 В.

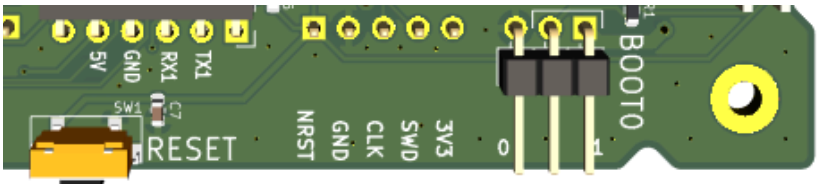
При необходимости можно задействовать пятый вывод для сброса микроконтроллера (пин RST на программаторе и NRST на плате FCM).

Перемычка BOOT0

На плате есть джампер **BOOT0**, который можно использовать, чтобы физически ввести микроконтроллер в режим загрузчика: в таком режиме

микроконтроллер ожидает, что в него будет загружена прошивка по одному из доступных для этого интерфейсов. Более подробную информацию о загрузчике STM32 можно найти в Application Note AN2606:

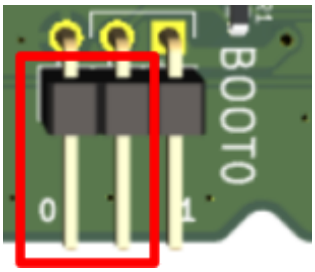
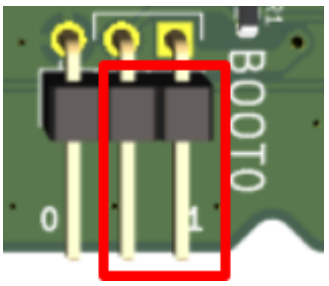
https://www.st.com/resource/en/application_note/an2606-introduction-to-system-memory-boot-mode-on-stm32-mcus-stmicroelectronics.pdf



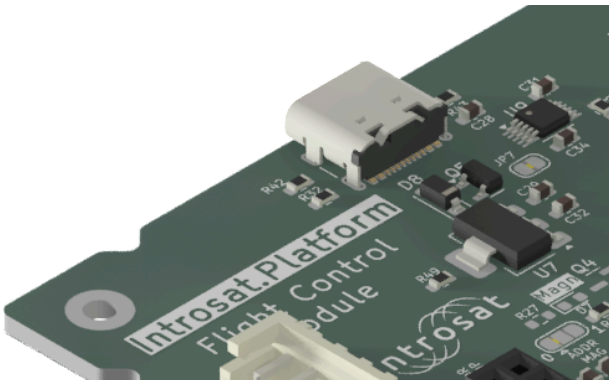
Джампер BOOT0 на обратной стороне платы полётного контроллера

Перед началом работы с платой полётного контроллера необходимо с помощью перемычки установить джампер **BOOT0** в положение **0** (то есть, объединить контакт с шелкографией 0 и средний контакт).

Для принудительного перевода микроконтроллера в режим загрузчика необходимо переставить перемычку в положение **1**. Если перестановка перемычки происходит при включенном питании, после этого нужно нажать на кнопку **RESET**. После загрузки прошивки желаемым способом необходимо вернуть перемычку в положение **0**, чтобы микроконтроллер мог выполнять загруженный на него код.

Обычный режим работы микроконтроллера	Микроконтроллер в режиме загрузчика
	

Подключение к разъёму USB Type-C



Разъём USB Type-C можно использовать в двух сценариях:

1. Подключение к блоку питания 5 В: таким образом можно обеспечить плату полётного контроллера питанием 3.3 В и 5 В (за вычетом падения напряжения на диоде Шоттки, около 0.2 – 0.4 В). Таким образом можно запитать только служебные модули и компоненты платформы, но не полезную нагрузку.
2. Подключение к USB-порту ПК: кроме обеспечения платы питанием, также происходит обмен данными по UART2.

Пины, используемые микроконтроллером как UART2:

- RX2 → PA3
- TX2 → PA2

Если при подключении к ПК материнская плата не отображается в диспетчере устройств и/или невозможно выбрать соответствующий COM-порт в мониторе порта, необходимо установить драйвер для CH340:

Для Windows:

https://acebottteam.github.io/acebott-docs-master/getting%20started/Arduino/Download%20CH340%20Driver%20on%20Windows%20System.html

Для MAC:

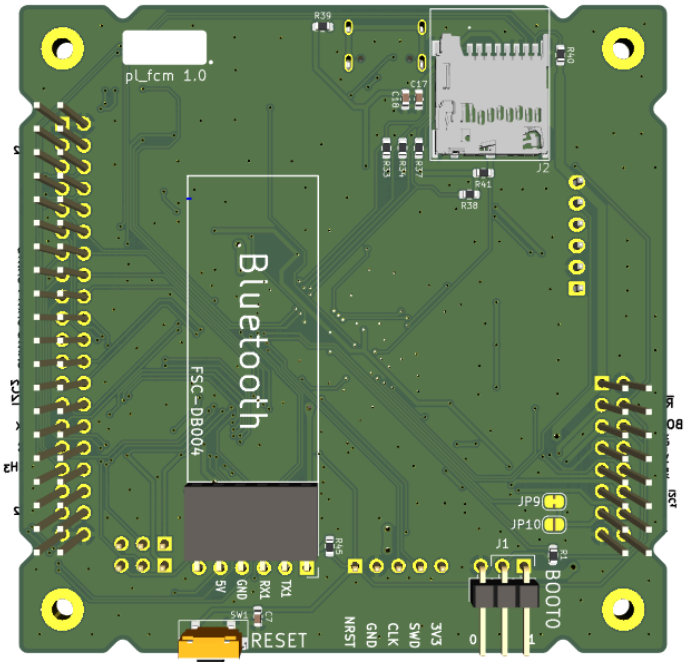
https://acebottteam.github.io/acebott-docs-master/getting%20started/Arduino/Download%20CH340%20Driver%20on%20MAC%20System.html

Гайд для различных ОС:

https://learn.sparkfun.com/tutorials/how-to-install-ch340-drivers/all

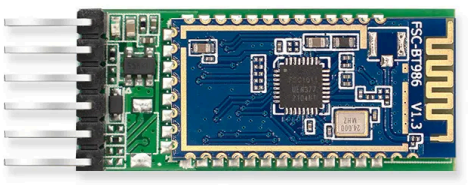
Обмен информацией по UART (основной канал связи)

Связь по Bluetooth



Обратная сторона платы полётного контроллера

С обратной стороны платы полётного контроллера есть разъём для подключения Bluetooth-модуля FSC-DB004-BT986. Модуль вставляется в разъём только одним образом, но перед подключением рекомендуется удостовериться, что шелкография контактов модуля соответствует шелкографии рядом с разъёмом на плате полётного контроллера (при этом RX1 на плате должен идти к TX модуля и наоборот, TX1 на плате к RX модуля).

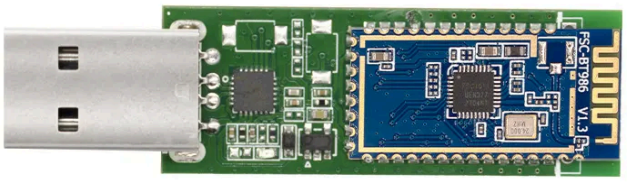


Bluetooth-модуль FSC-DB004-BT986

На разъём для подключения Bluetooth выведен UART1 микроконтроллера, а конкретно следующие пины:

- RX1 → PA10
- TX1 → PA9

В качестве ответной части к ПК подключается Bluetooth-модуль с встроенным USB-UART конвертером FSC-DB005-BT986.



FSC-DB005-BT986

Настройка Bluetooth-модулей описана в разделе ниже.

Подключение USB-UART конвертера

Для обмена информацией с микроконтроллером по UART1 к разъёму для подключения Bluetooth также можно можно подключить USB-UART конвертер, используя три провода “мама-папа”.



Подключение следует производить по следующей схеме:

FCM	USB-UART конвертер
TX1	RXD
RX1	TXD
GND	GND

USB-UART конвертер будет питаться при этом от USB порта ПК, а плату

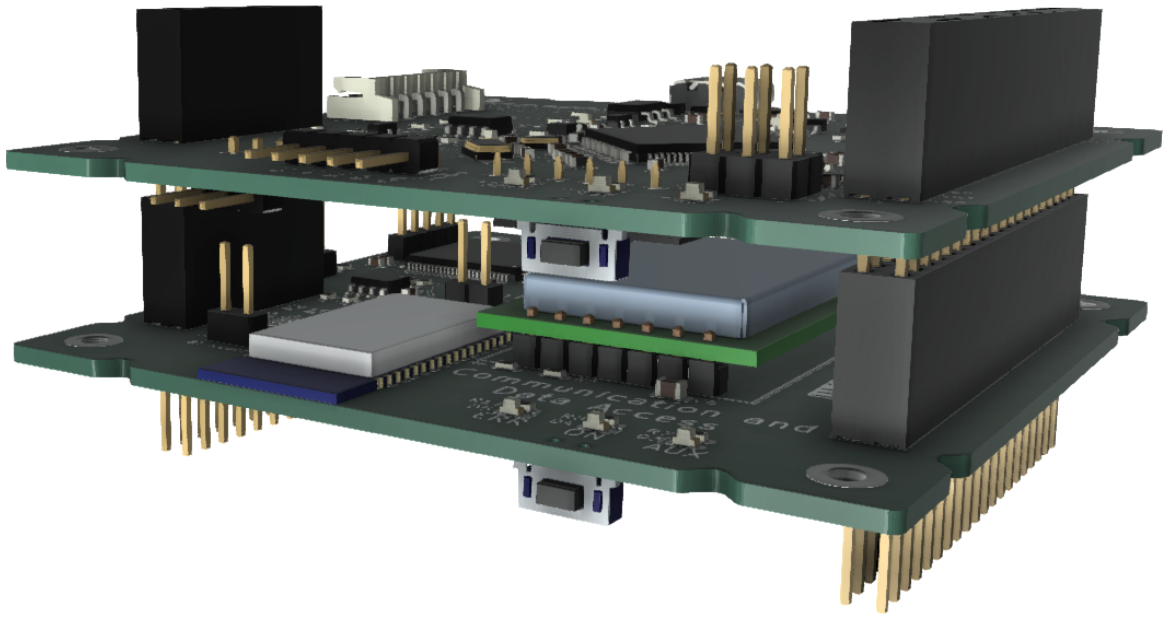
полётного контроллера необходимо обеспечить питанием от платы питания, разъёма USB Type-C или программатора.

Радиосвязь

При соединении платы полётного контроллера и платы связи в стек становится возможным обмен данными по радиосвязи. UART1 платы полётного контроллера подключается к плате связи через служебную шину IntroBus_S; при этом используются следующие пины:

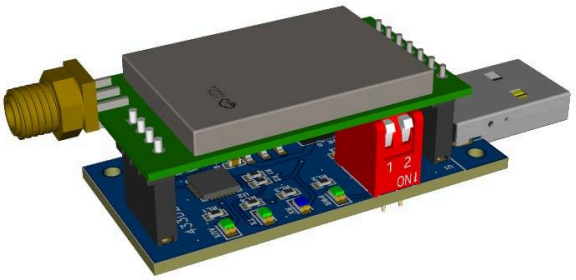
- RX1 → PA10
- TX1 → PA9

Вся информация, передаваемая по UART1, в таком случае поступает на радиомодуль LoRa и распространяется в виде радиосигнала на частоте, установленной при настройке платы связи.



Соединение платы полётного контроллера (FCM) и платы связи (CDAM)

В качестве ответного устройства выступает модуль LoRa-UART, подключаемый к USB-порту ПК.

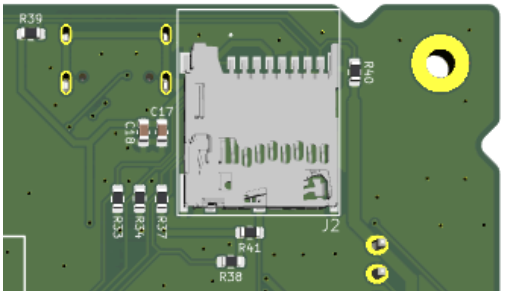


Модуль LoRa-UART

Для обмена данными радиомодули LoRa на плате связи и модуле LoRa-UART должны быть настроены одинаково. Подробнее о настройке модуля LoRa-UART можно прочитать в разделе ниже.

Внимание!
При работе с платой связи и модулем LoRa-UART убедитесь, что к ним подключена антенна. Без антенны модули LoRa могут выйти из строя.

SD-карта



Слот для подключения microSD карты

Для логгирования к плате полётного контроллера подключается MicroSD карта. Для подключения используется протокол SDMMC со следующими пинами:

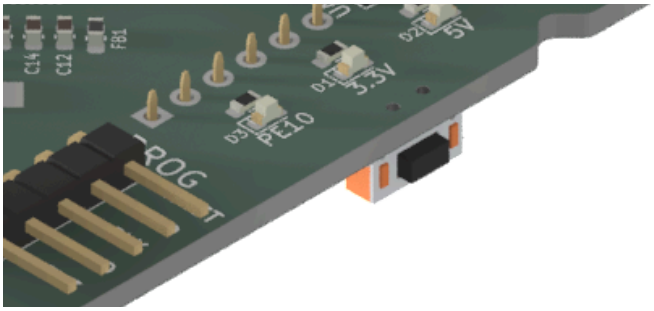
- D0 → PC8 (первый бит данных)
- D1 → PC9 (второй бит данных)
- D2 → PC10 (третий бит данных)
- D3 → PC11 (четвёртый бит данных)
- CMD → PD2 (передача команд и ожидание ответа)
- CK → PC12 (тактирование)

- CD → PB8 (detect pin, проверка подключения карты)

MicroSD карта вставляется в слот до щёлчка. Чтобы вытащить карту, на неё нужно предварительно нажать. Если нет щелчка при нажатии на карту, попробуйте просто потянуть её на себя.

Когда microSD вставлена в слот, пин CD, подтянутый к 3.3 В, физически подключается к земле платы: таким образом можно отслеживать, вставлена ли карта в слот. Подробнее о работе с microSD картой по протоколу SDMMC можно прочитать в разделе ниже.

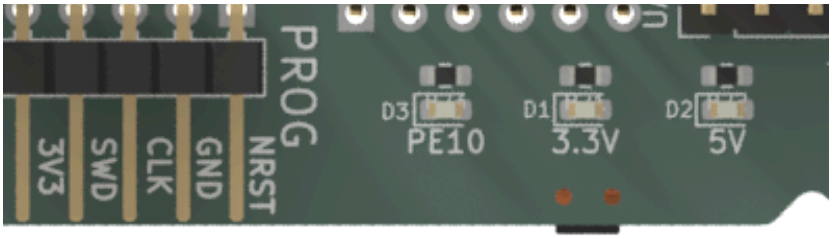
Кнопка RESET



Кнопка RESET на плате полётного контроллера

Кнопка RESET служит для перезагрузки микроконтроллера. После её нажатия выполнение загруженной на микроконтроллер программы начинается заново.

Светодиоды на плате



Светодиоды на плате полётного контроллера

На плате полётного расположены три светодиода:

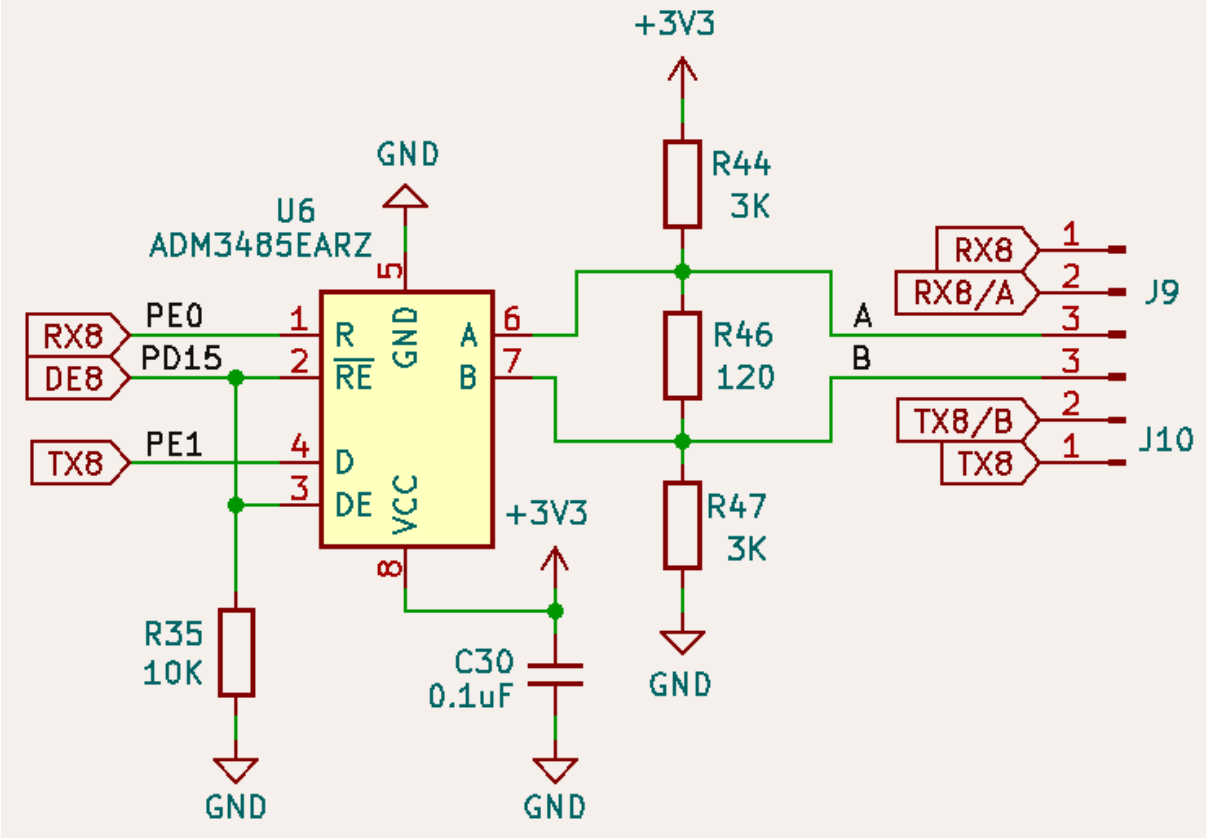
1. Пользовательский светодиод, подключенный к пину PE10 микроконтроллера
2. Светодиод, сигнализирующий о наличии бортового питания 3.3 В
3. Светодиод, сигнализирующий о наличии бортового питания 5 В

Обратите внимание, что последние два светодиода не дают информации о том, подаётся ли в данный момент питание на полезную нагрузку. Управление каналами 3.3 В и 5 В для питания полезной нагрузки происходит на модуле электропитания.

Установка режима работы пинов RX8/A и TX8/B

На плате полётного контроллера установлен драйвер интерфейса RS485, позволяющий использовать интерфейс UART8 как RS485. Для этого задействуется дополнительный пин микроконтроллера.

- RX8 (UART8) – R (RS485) → PE0
- TX8 (UART8) – D (RS485) → PE1
- DE8 (RS485) → PD15

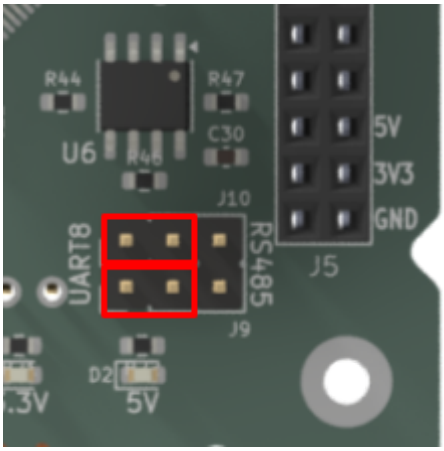
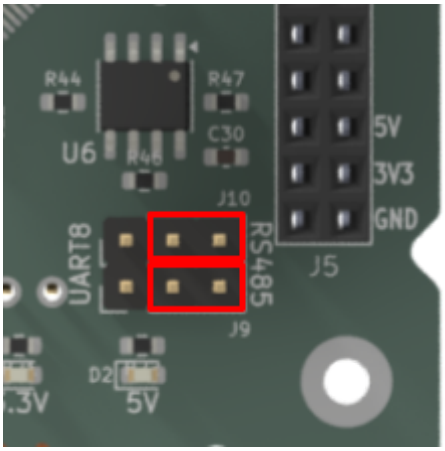


Часть схемы электрической принципиальной (подключение драйвера RS485)

На выходе драйвера, если он включен с помощью управляющего сигнала DE8, формируются два сигнала: A и B. По этим линиям осуществляется двусторонний обмен информацией с внешним устройством, имеющим интерфейс RS485.

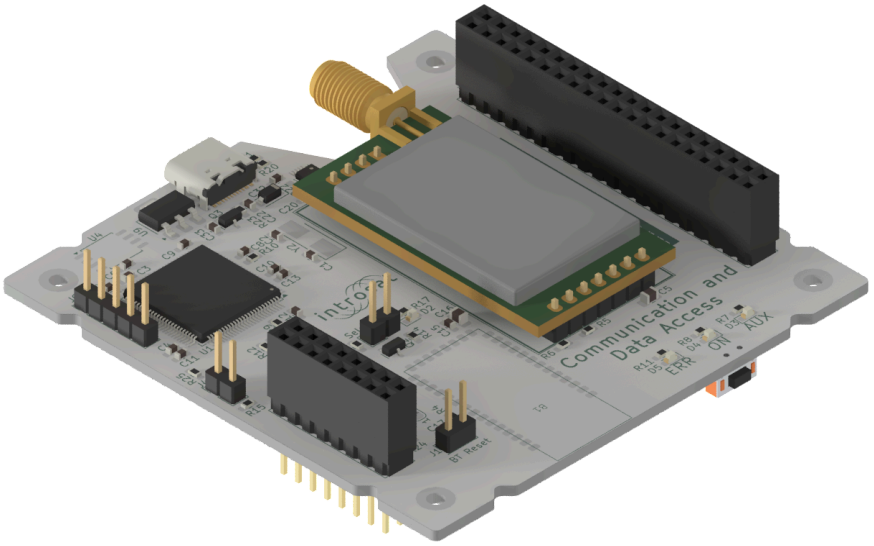
На шине IntroBus_L есть два контакта: RX8/A и TX8/B, режим работы которых выбирается с помощью перемычек J9 и J10.

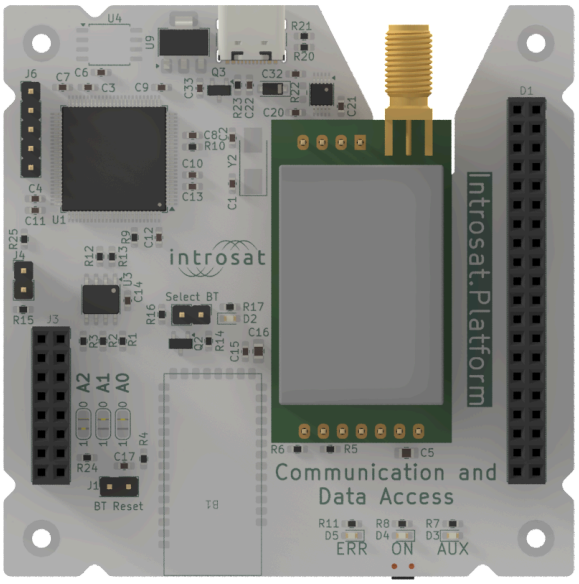
Для использования контактов RX8/A и TX8/B в качестве UART8, установите перемычки J9 и J10 в положение **UART8**. Для использования интерфейса RS485 установите перемычки в положение **RS485**.

Режим UART8	Режим RS485
	

Плата связи (Communication and Data Access Module)

Общая информация





Внешний вид платы связи

Плата связи позволяет обмениваться данными с наземной станцией с помощью радиомодуля, работающего по протоколу LoRa.

Габариты платы	76x76 мм
Расстояние между отверстиями для крепления	65 мм

Модель микроконтроллера	STM32H750VBT6
Напряжение питания микроконтроллера	3,3 В
Радиомодуль	Ebyte E32-433T30D

Кроме микроконтроллера и радиомодуля на плате также установлен датчик температуры LM75AD, его адрес на шине I2C - 0x4C. Шина I2C, к которой подключён этот датчик, соответствует шине I2C1 на плате полётного контроллера.

Радиомодуль на плате работает в диапазоне частот 410-441 МГц, при этом частоту можно выставить с шагом в 1 МГц. Это позволяет использовать 32 канала не мешая друг другу. Выбор канала должен основываться, в том числе, на зашумлённости тех или иных частот в точке запуска.

Передача данных по радиоканалу без подключённой антенны может повредить радиомодуль. Рекомендуется всегда подключать антенну.

Немного о передаче данных

Данные на микроконтроллере хранятся в переменных - в памяти в двоичном виде. Чтобы передать эти данные по радиоканалу, необходимо преобразовать нули и единицы в некоторые колебания электромагнитного поля. Это преобразование происходит на модуле E32-433T30D. Он использует протокол LoRa(Long Range) для передачи данных. Хотя этот протокол и позволяет передавать данные на расстояние до нескольких десятков километров, у него есть свои недостатки. В первую очередь - низкая скорость передачи - то есть количество бит, передаваемых в секунду. Стандартное значение этого параметра для используемого модуля - 2.4 кбод (2400 бит в секунду). То есть радиомодуль не может отправлять более 2400 бит каждую секунду. Если на вход ему подавать 4000 бит каждую секунду, то он не будет успевать их передать, и часть данных будет теряться.

Плата связи реализует дополнительную буферизацию данных: ей можно передать до 4096 на высокой скорости, после чего она будет постепенно отправлять их на радиомодуль. Это позволяет упростить отправку данных. Несмотря на это, следует иметь ввиду некоторые особенности работы радиомодуля.

Особенности радиомодуля
Модуль E32-433T30D отправляет данные пакетами по 58 байт. Разбиение на эти пакеты происходит на самом модуле, отдельно реализовывать это разбиение не требуется. Однако, после отправки каждого такого пакета есть некоторая задержка. То есть, в то время как разница во времени передачи набора данных из 57 байт и 58 байт будет относительно мала, разница во времени передачи набора данных из 58 байт и 59 байт уже будет заметна.

Формирование пакета данных
Учитывая все вышеперечисленные особенности, правильным решением будет основательно проработать структуру данных, которые вы собираетесь передавать по радиоканалу. Не стоит передавать символьные строки, лучше передавать только данные. Рассмотрим пример: необходимо передать по радиоканалу угловую скорость по 3 осям. Самым простым способом это можно сделать, передав подобную строку:

```
gx:0.12;gy:0.315;gz:0.023
```

(Значения подставлены в строку, например с помощью `sprintf()`)

Эта строка состоит из 25 символов, то есть 25 байт. В то же время, если значение по каждой из осей хранится в переменной типа `float` (занимает 4 байта), то суммарное количество полезных данных, которые мы передаём, равно 3*4=12 байт. А преобразование в строку ещё и теряет точность, которую имеет `float`.

Таким образом мы передаём 13 лишних байт, ещё и теряем точность. Лучше будет передавать только сами значения, побайтово. Это усложняет обработку принятых данных, ведь на принимающей стороне вместо красивого читаемого текста будет лишь набор байт, который ещё надо преобразовать в число с плавающей точкой. Однако это очень сильно ускоряет передачу данных.

В процессе отладки программы может быть удобнее передавать данные в виде строк, но важно в какой-то момент перейти в более разумному использованию радиоканала, передавать только данные. Для этого можно использовать преамбулу(байт(ы), который(е) встречается только в начале пакета данных), постамбулу(байт(ы), который(е) встречаются только в конце пакета). Если планируется передача пакетов нескольких типов (например: общая телеметрия, состояние полезной нагрузки, расширенная телеметрия), имеет смысл добавить в начало пакета дополнительный байт, отвечающий за тип пакета.

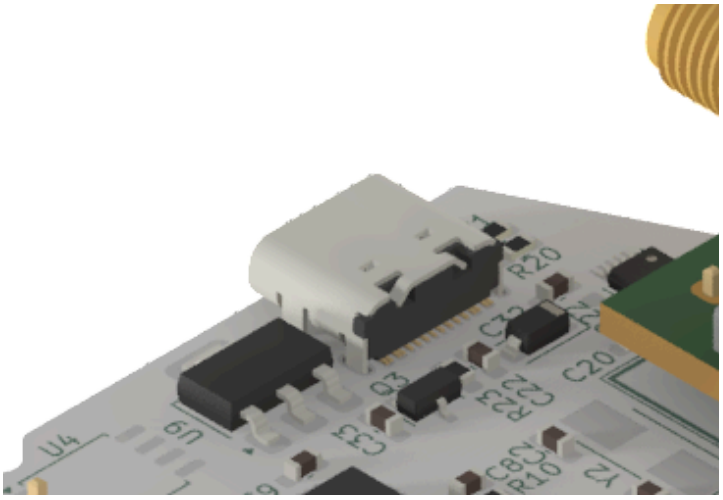
Приём и передача данных
Плата связи подключена к одному из UART-интерфейсов в шине IntroBus-S. На плате FCM - это UART1.

Для передачи данных достаточно отправить в этот UART данные с платы FCM. После того, как плата связи передаст данные по радиоканалу, она вернёт в UART символ **0x06** (ACK из таблицы ASCII). После этого можно передавать следующий набор данных. Эти наборы данных должны быть менее 4096 байт, иначе часть данных будет потеряна.

При передаче данных на аппарат с наземной станции, плата связи сначала принимает все пришедшие подряд пакеты (помним, что радиомодуль для передачи разбивает данные на пакеты по 58 байт. Значит, если отправить с земли 100 байт, будет отправлено два пакета: 58 и 42 байта), записывает их во внутренний буфер, после чего, при отсутствии следующего пакета в течение некоторого короткого времени, отправляет все принятые данные в UART. Эти данные можно можно принять на плате FCM из UART1.

Настройка радиомодуля LoRa
Плата связи позволяет изменять основные настройки радиопередатчика с помощью AT-команд. Для этого необходимо подключиться к плате связи через провод USB Type-C и открыть последовательный порт на скорости **115200** бод.

При настройке платы связи рекомендуется отключать её от стека плат, чтобы исключить возможность наличия нескольких источников питания



Разъём USB Type-C на плате связи

Ппосле подключения плату можно настроить следующими командами:

Проверка соединения: AT+PING
Эта команда позволяет проверить, успешно ли произошло соединение. Если всё хорошо, в ответ придёт:

→ `AT+PING?`

← `+PONG`

Настройка мощности передачи: AT+LTXPWR
Значения мощности передачи:

- `0` - 30 дБм
- `1` - 27 дБм
- `2` - 24 дБм
- `3` - 21 дБм

Запрос мощности передачи:

→ `AT+LTXPWR?`

Ответ:

← `+LTXPWR=X` , где `X` - установленная мощность

Установка мощности передачи:

→ `AT+LTXPWR=X`

Ответ:

← `+OK`

Настройка скорости передачи: AT+LAIRRATE

Значения скорости передачи данных по радиоканалу:

- `0` - 300 бод
- `1` - 1200 бод
- `2` - 2400 бод
- `3` - 4800 бод
- `4` - 9600 бод
- `5-7` - 19200 бод

Запрос скорости передачи:

→ `AT+LAIRRATE?`

Ответ:

← `+LAIRRATE=X` , где `X` - установленная скорость

Установка скорости передачи `X`:

→ `AT+LAIRRATE=X`

Ответ:

← `+OK`

Настройка канала LoRa: AT+LCHANNEL

Доступны каналы `1-31`.

> Обратите внимание, что частота канала `C` равняется `410 + C` МГц, где `C` - номер канала

Запрос канала:

→ `AT+LCHANNEL?`

Ответ:

← +LCHANNEL=X , где X - установленный канал

Установка канала X :

→ AT+LCHANNEL=X

Ответ:

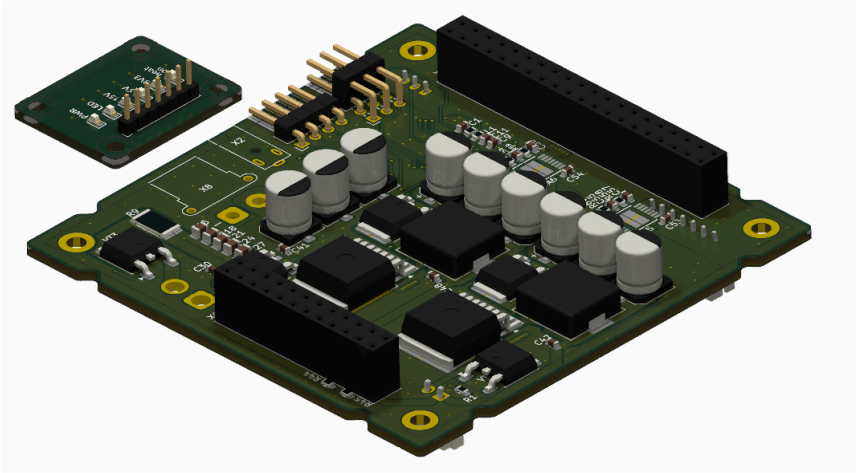
← +OK

Если в ответ на команду модуль присылает
← +ERROR

следует проверить правильность отправленной команды и параметров.

Модуль электропитания (Power Module)

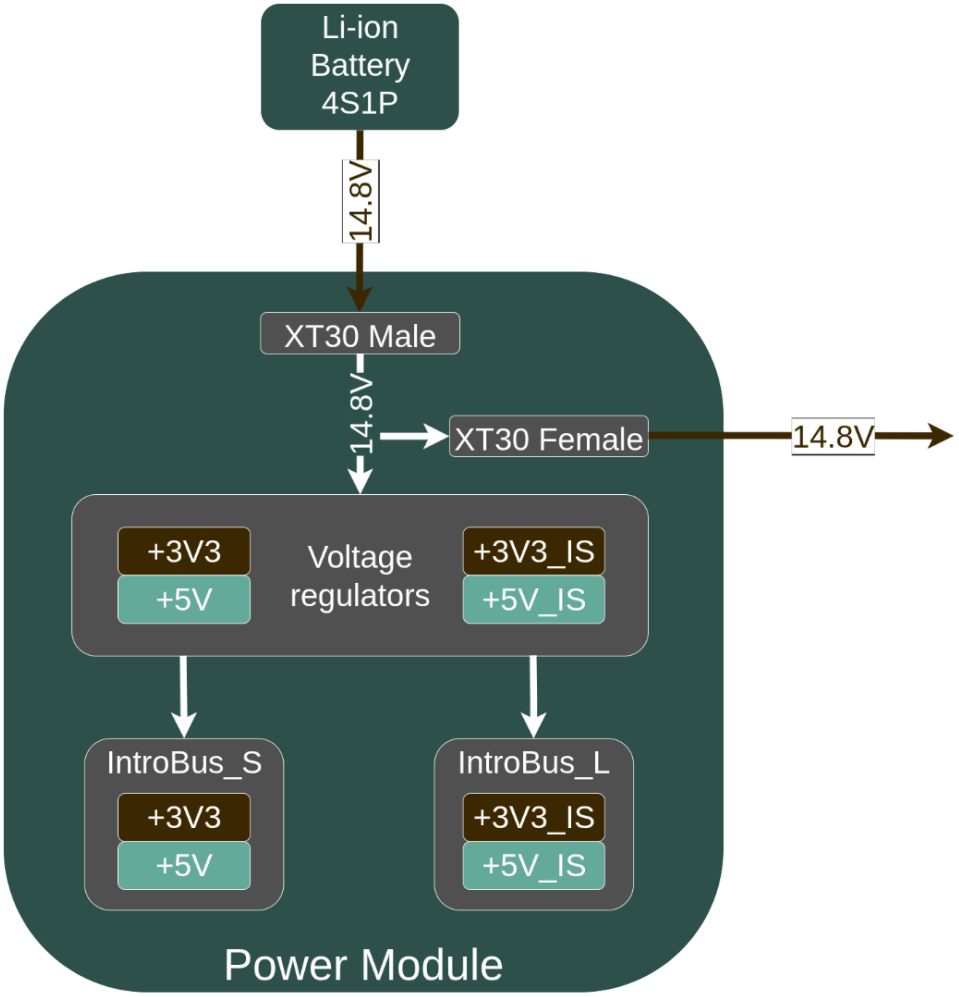
Общая информация



Внешний вид платы электропитания (модуль индикации отдельно)

Модуль электропитания обеспечивает питанием конструктор IntroSat.Platform. Плата модуля электропитания всегда находится вверху стека. Источником питания может выступать сборка Li-ion аккумуляторов 4S1P (4 последовательно соединённых аккумулятора) с номинальным напряжением **14.8 В**, либо источник питания **12 В**, подключаемый к электрической сети.

Внимание!
Сборка аккумуляторов с подогревом выдаётся на турнире “Орбита”. При подготовке используется источник питания 12 В.



Упрощённая функциональная схема модуля электропитания

Входное напряжение поступает на разъём **XT30-Male** (14.8 В с аккумуляторной сборки или 12 В с источника питания) и далее разделяется на несколько каналов:

- идёт без преобразований и стабилизации на разъём **XT30-Female** для питания устройств полезной нагрузки, требует включения вручную
- стабилизуется до напряжения 5 В и делится на два независимых канала:
    - +5V – бортовое питание, идёт на шину IntroBus_S, всегда включено
    - +5V_IS – для питания полезной нагрузки, идёт на шину IntroBus_L, требует включения вручную

- стабилизируется до напряжения 3.3 В и делится на два независимых канала:
    - +3V3 – бортовое питание, идёт на шину IntroBus_S, всегда включено
    - +3V3_IS – для питания полезной нагрузки, идёт на шину IntroBus_L, требует включения вручную

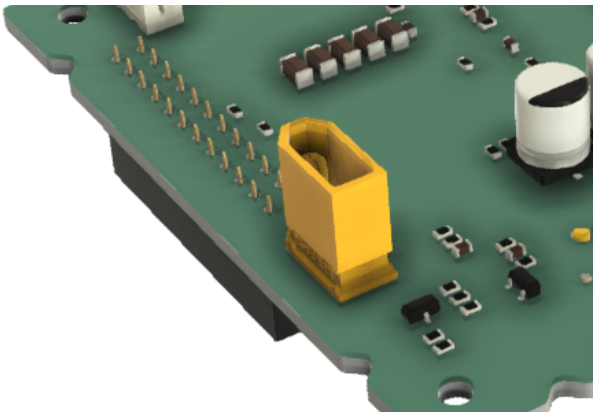
Характеристики каналов питания

При проектировании эксперимента необходимо учитывать ограничения по току, наложенные на используемые для подключения полезной нагрузки каналы питания. При превышении указанных в таблице ниже токов каналы будут автоматически отключены.

Внимание!
Не допускайте короткого замыкания и превышения указанных токов во избежание выхода устройств из строя.

Канал питания	**Максимальный ток нагрузки, А**
Нестабилизированное напряжение 14.8 В	6
Стабилизированное напряжение 5 В	3
Стабилизированное напряжение 3.3 В	3

Подключение источника питания



Разъём для подключения источника питания

Разъём XT30-Male, подписанный шелкографией как **X1**, используется для подключения источника питания.

Подключение источника питания 12 В необходимо производить через переходник на XT30-Female.



Источник питания 12 В

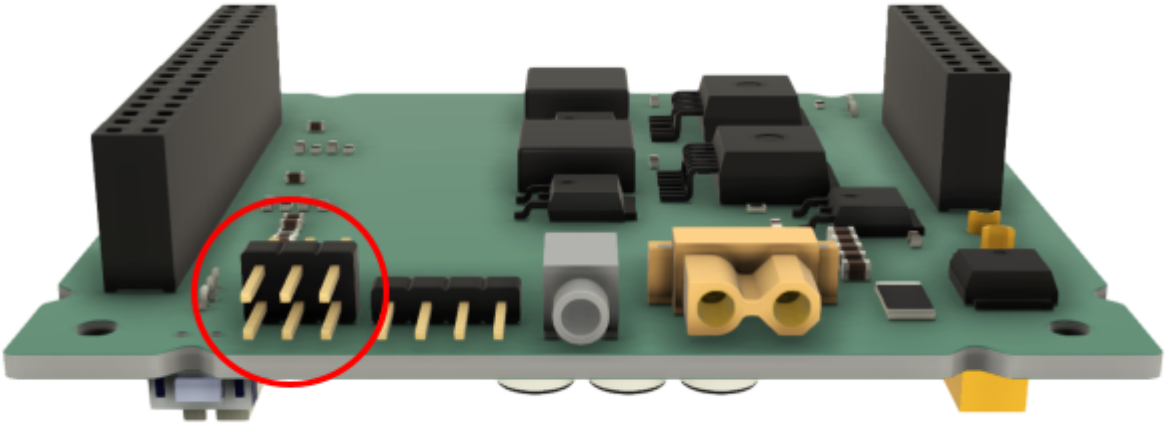


Переходник для источника питания на XT30

Внимание!
Не используйте линии питания 3.3 В и 5 В на программаторе St-Link, USB-UART конвертере и других внешних устройствах, подключаемых к конструктору, когда в стек подключен модуль электропитания.

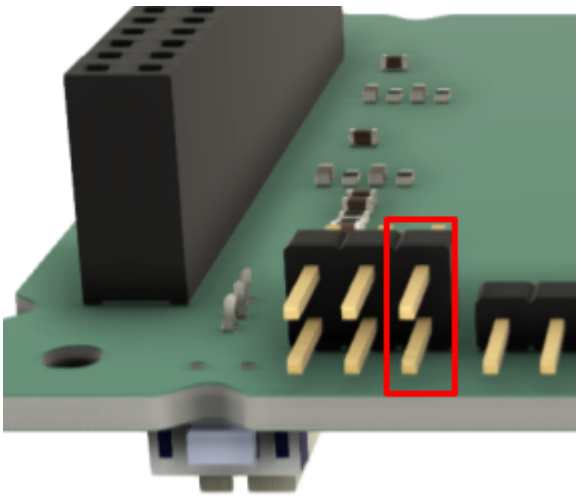
Включение каналов питания

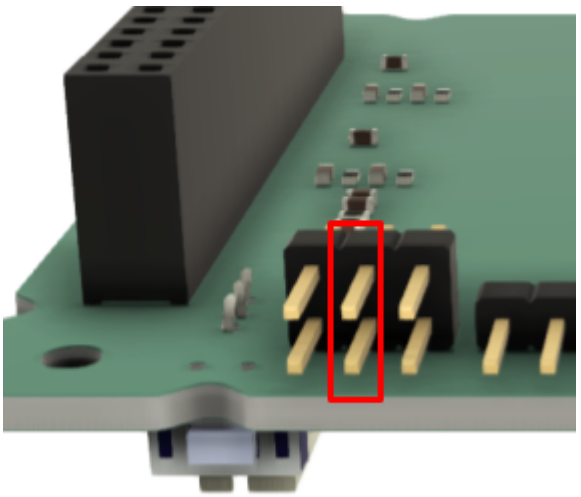
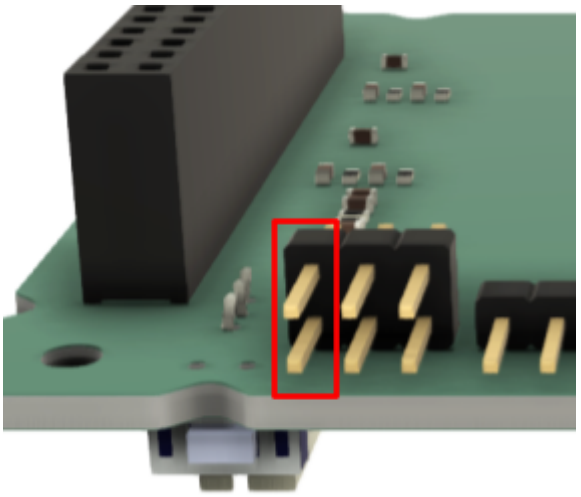
Включение каналов питания для полезной нагрузки происходит с помощью установки перемычек на шестиконтактный разъём **X6**. Перемычки устанавливаются перпендикулярно плате.



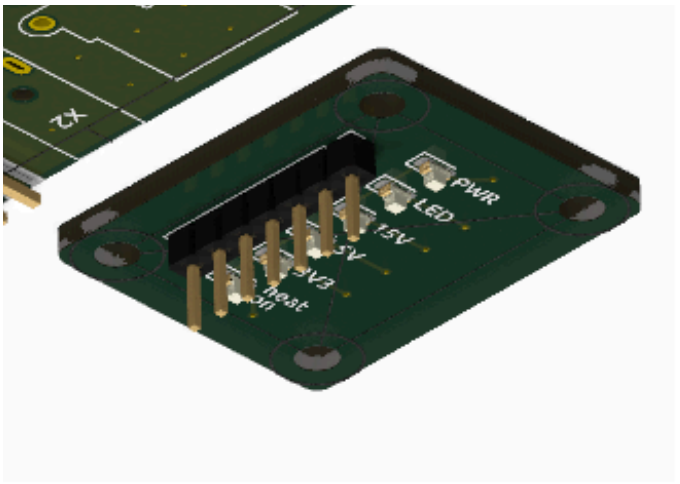
Разъём X6, отвечающий за включение каналов

В таблице ниже указано соответствие между каналом питания и контактами, которые требуется замкнуть для его включения:

Канал	Расположение перемычки
3.3 В (+3V3_IS)	

5 В (+5V_IS)	
Нестабилизированное 14.8 В	

При включении канала загорается соответствующий светодиод на модуле индикации, соединённым с платой электропитания.

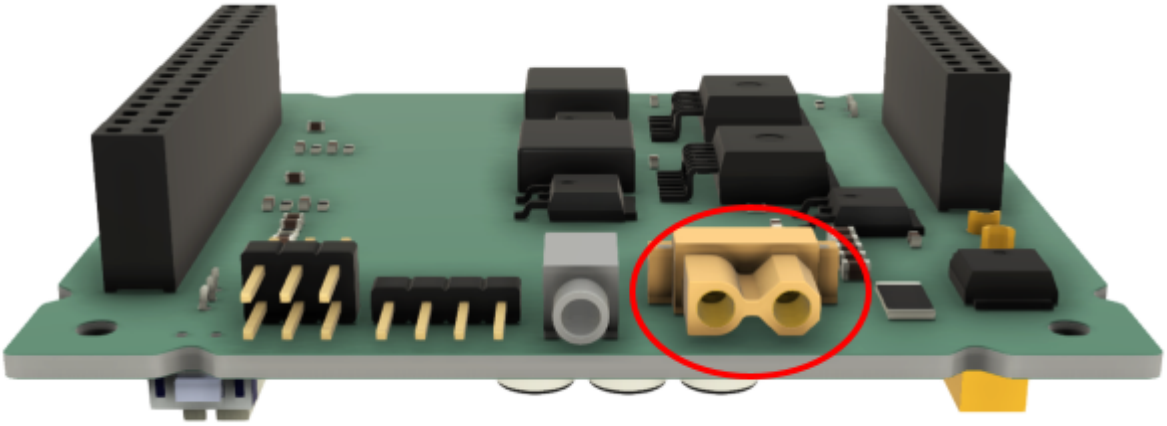


Модуль индикации

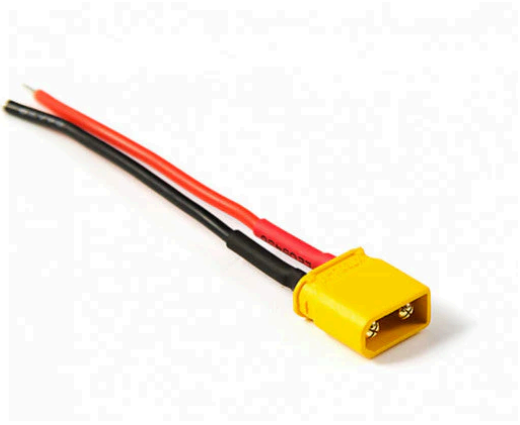
Подключение питания полезной нагрузки

Напряжения 3.3 В и 5 В для питания полезной нагрузки поступают на плату полётного контроллера и интерфейсную плату насквозь через разъёмы шины IntroBus_L. К ним можно подключиться проводами для беспаечной сборки на этапе макетирования, а перед полётом обеспечить соединение пайкой.

Подключение к нестабилизированному напряжению 14.8 В осуществляется через угловой разъём XT30-Female на модуле питания (**X8**) посредством кабеля с разъёмом XT30-Male, идущего в комплекте. Вторая часть шлейфа припаивается к устройству, которое требуется обеспечить питанием (красный провод – плюс питания, чёрный – земля).



Разъём X8 на модуле питания (угловой XT30-Female)



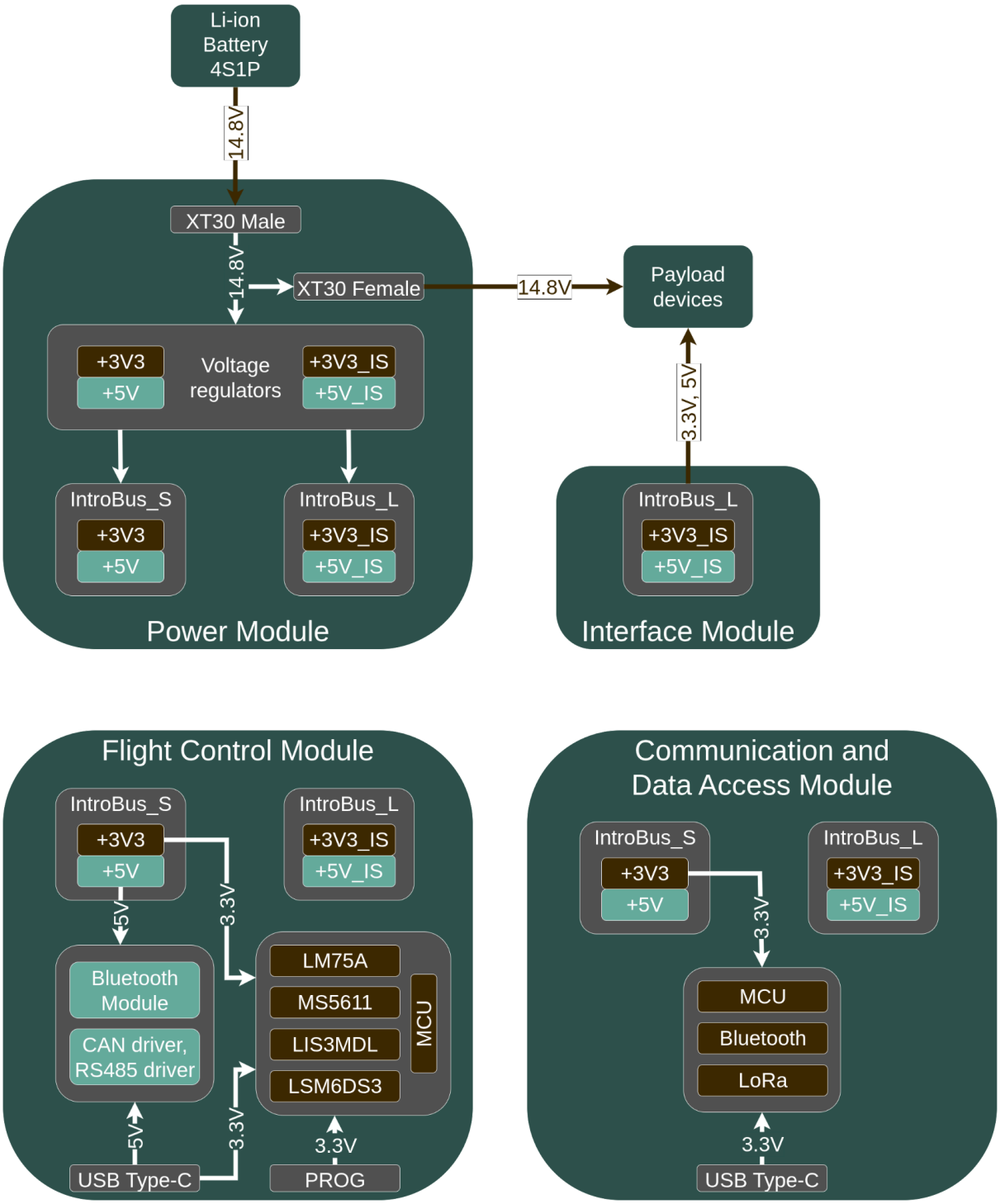
Кабель для подключения питания 14.8 В к устройствам полезной нагрузки

Обратите внимание, что напряжение 14.8 В не стабилизировано – это номинальное напряжение из расчёта, что четыре аккумулятора подключены последовательно. Максимальное напряжение на каждом

аккумуляторе равно 4.2 В, и они могут разряжаться до напряжения около 3.2 В. Если для питания устройства полезной нагрузки требуется конкретное напряжение, необходимо продумать подключение питания через дополнительный внешний стабилизатор.

При использовании источника питания 12 В напряжение на данном разъёме будет стабильно, но при запуске используются аккумуляторы.

Особенности схемы питания



Функциональная схема питания конструктора IntroSat.Platform

При соединении служебных модулей конструктора в стек (модуля электропитания, платы связи, платы полётного контроллера и интерфейсного модуля) питание проходит насквозь по шине IntroBus:

- на шине IntroBus_S расположены контакты 3.3 В и 5 В для питания самих служебных модулей
- на шине IntroBus_L расположены контакты 3.3 В и 5 В (на функциональной схеме обозначены как +3V3_IS и +5V_IS) для питания полезной нагрузки

Также возможно использовать служебные платы отдельно от стека:

- плату связи можно питать через разъём USB Type-C (бортовое напряжение 3.3 В)
- модуль полётного контроллера можно питать через разъём USB Type-C (бортовые напряжения 3.3 В и 5 В)
- бортовое напряжение 3.3 В на модуле полётного контроллера можно получить через программатор ST-Link

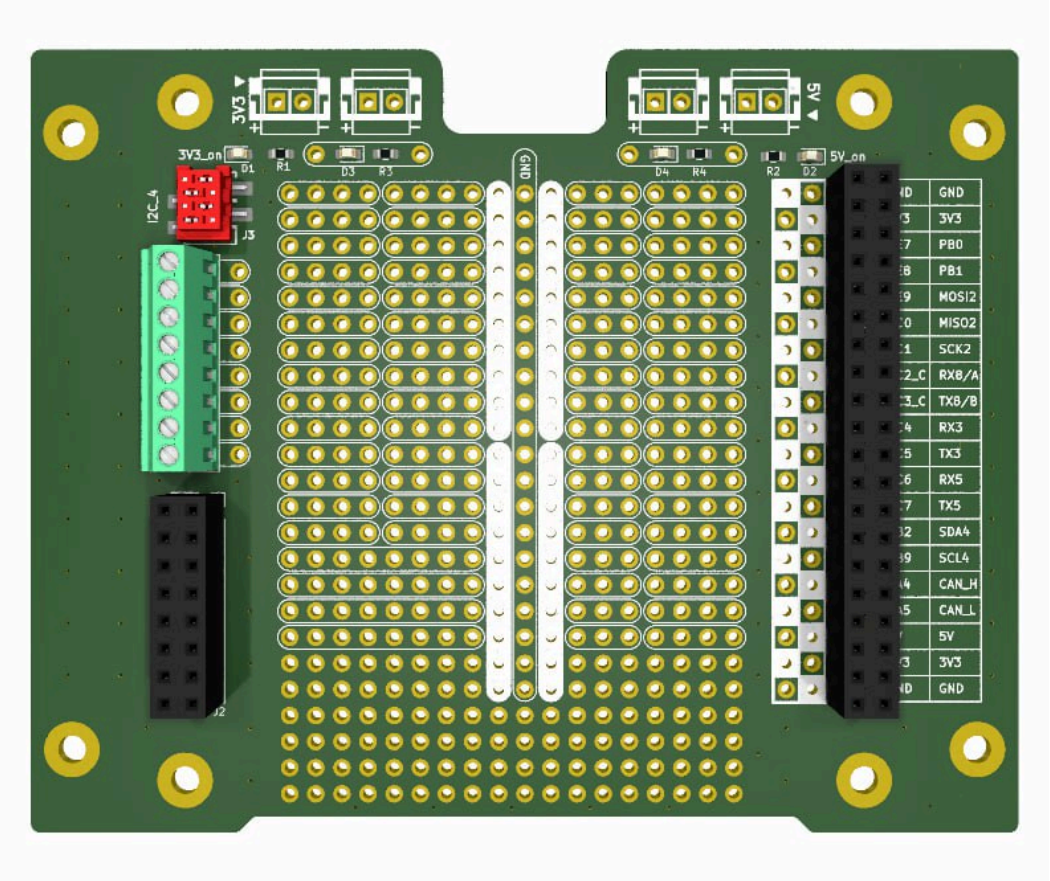
В случае любого из вышеописанных трёх сценариев полученное напряжение передаётся к другим платам через шину IntroSat_S: например, если к модулю полётного контроллера подключен программатор, напряжение 3.3 В также будет присутствовать на плате связи. Обратите внимание, что этими способами можно получить только бортовое питание, но не питание полезной нагрузки.

Внимание!
Если несколько плат соединены в стек, используйте только один источник питания.

Отключайте контакт 3.3 В программатора ST-Link от платы полётного контроллера, если питание подаётся через разъёмы USB Type-C или модуль электропитания.

Интерфейсный модуль

Общая информация



Внешний вид интерфейсного модуля

Интерфейсный модуль предназначен для подключения полезной нагрузки. На этапе макетирования подключение можно производить через шину IntroBus_L с использованием проводов для беспаечной сборки.

При подготовке к стратосферному запуску все провода должны быть надёжно соединены с интерфейсным модулем с использованием пайки или разъёмов с фиксацией (JST, micromatch, клеммники).

Особенности интерфейсного модуля:

1. Контактные площадки для сквозного монтажа на плате расположены с шагом 2.54 мм.
2. Шелкографией на плате показаны соединения: контактные площадки для сквозного монтажа объединены в группы по три и четыре контакта.

3. Посередине платы проходит линия земли – контактные площадки соединены и подписаны GND.
4. По обе стороны от земли посередине платы расположены длинные ряды объединённых контактов. Их удобно подключить к линиям питания 3.3 В и 5 В или к другим повторяющимся в использовании сигналам.
5. На плате расположено по две пары контактных площадок, подписанных как 3V3 и 5V с указанием плюса и минуса питания. Туда приходят соответствующие напряжения питания полезной нагрузки.
6. Контакты шины IntroBus_S не выведены на плату. Их использовать нельзя.
7. Контакты шины IntroBus_L подписаны шелкографией в таблице рядом с разъёмом. С другой стороны от разъёма расположены контактные площадки для сквозного монтажа: к ним можно припаиваться напрямую, либо пробрасывать перемычки на свободные контактные площадки интерфейсной платы.
8. На плате интерфейсного модуля присутствует разъём micromatch, куда выведен интерфейс I2C4 платы полётного контроллера. Если на устройстве полезной нагрузки, подключаемом по I2C, есть разъём micromatch, можно соединить его с интерфейсным модулем шлейфом micromatch.
9. На плате интерфейсного модуля расположен 8-контактный клеммник. Его контакты можно соединить с требуемыми контактами шины IntroBus_L пайкой.
10. На интерфейсном модуле расположены светодиоды, сигнализирующие о наличии напряжения питания полезной нагрузки: 3V3_ON и 5V_ON.
11. Также на плате присутствуют пользовательские светодиоды, к которым последовательно подключены токоограничивающие резисторы: их можно использовать по своему усмотрению, для индикации событий или отладки работы полезной нагрузки.

Создание сборно-разборных соединений с полезной нагрузкой
В набор конструктора IntroSat.Platform входит дополнительная коммутация:

- двухконтактные клеммники с шагом 2.54 мм 
- разъёмы JST B2B-EH-A (Male) 
- PLS-2 (имеет шаг 2.54 мм) 

Так как расстояние между контактными площадками для сквозного монтажа составляет 2.54 мм, все эти разъёмы можно напаивать на плату интерфейсного модуля для создания сборно-разборных соединений с полезной нагрузкой. Клеммники и JST разъёмы (но не PLS-2) можно использовать в стратосферном запуске, так как они обеспечивают достаточную степень фиксации при соединении.

С клеммниками всё достаточно просто: зачищенные провода вставляются в клеммник, и контакт прижимается при закручивании отвёрткой.

Разъёмы JST требуют ответного (Female) разъёма для подключения. Для этого в комплекте также есть несколько кабелей JST “Female to Female”: можно разрезать их пополам, зачистить провода и припаять к полезной нагрузки провода, либо, если на устройстве полезной нагрузки есть контакты для сквозного монтажа, можно также припаять к ним разъёмы JST “Male”.

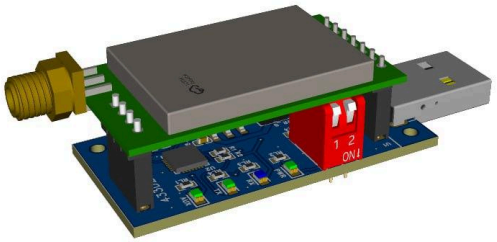


JST “Female to Female” cable

Совет.
Для подключения устройств полезной нагрузки рекомендуется создавать сборно-разборные соединения с использованием клеммников, разъёмов

JST и кабелей. В таком случае, если одно из устройств будет некорректно работать или выйдет из строя, будет легко локализовать место ошибки, последовательно отключая и подключая устройства к интерфейсному модулю.

Модуль LoRa-UART



Внешний вид модуля LoRa-UART

Модуль LoRa-UART подключается к USB-порту ПК. Если радиомодуль LoRa получает сигнал от ответного модуля, установленного на спутнике, данные можно будет увидеть в мониторе порта.

Внимание!
Запрещается подключать модуль LoRa-UART к ПК без подключенной антенны!

Перед работой с модулем необходимо подключить к его SMA-разъёму антенну 433 МГц.

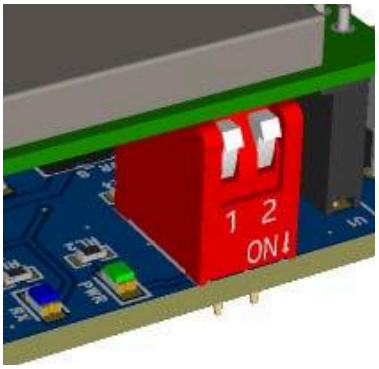


Антенна 433 МГц с ответным SMA-разъёмом

Модуль имеет два основных режима работы:

1. Sleep mode (M0 = 1, M1 = 1) – используется для настройки модуля
2. Normal mode (M0 = 0, M1 = 0) – в этом режиме осуществляется приёмопередача

Переключение между режимами осуществляется изменением положения переключателей на модуле.



Переключатели для изменения режима работы радиомодуля LoRa

Режим работы	**Положение переключателей**
Режим приёмопередачи (normal mode)	Оба переключателя вверху
Режим настройки (sleep mode)	Оба переключателя внизу, в положении ON

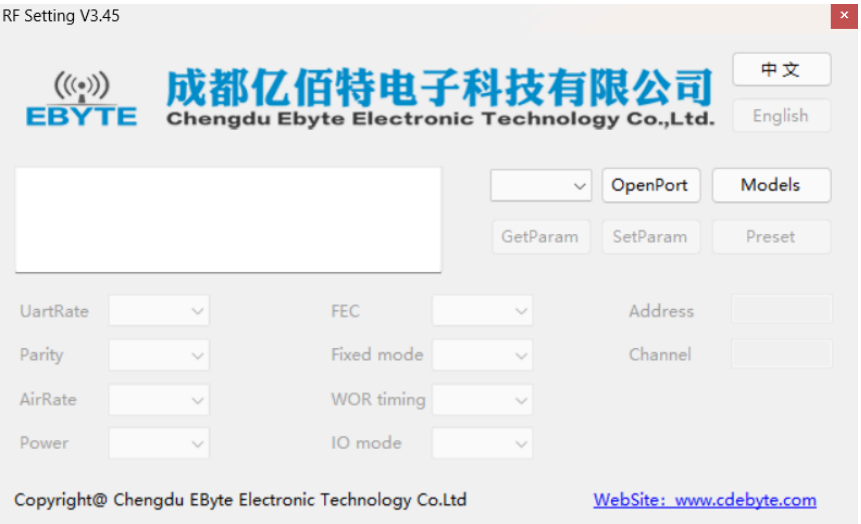
Для обмена данными между двумя радиомодулями LoRa они должны иметь одинаковые настройки.

Настройка радиомодуля LoRa
Модель LoRa – E32-433T30D. Более подробную информацию можно найти в datasheet на модуль:

https://www.cdebyte.com/pdf-down.aspx?id=1462

Настройка радиомодуля LoRa осуществляется с помощью программы RF Setting от производителя модуля EBYTE. Скачайте её по ссылке:

http://74.117.156.195/verify.php?arquivo=RFSettingv3-45.zip



Внешний вид программы RF Setting

Если вы не знаете китайского, переключите язык на английский, нажав кнопку **English**.

В первую очередь нужно выбрать COM-порт, к которому подключена LoRa. Выберите порт и нажмите кнопку **OpenPort**. Если программа пишет, что на данном COM-порте модуль не обнаружен, возможно, необходимо установить драйвер для USB-UART конвертера CP2102. Его можно скачать по ссылке:

https://www.silabs.com/software-and-tools/usb-to-uart-bridge-vcp-drivers?tab=downloads

Если порт успешно открыт, нажмите кнопку **GetParam** – в окне появятся параметры, на которые настроена LoRa сейчас. После изменения параметров нужно нажать **SetParam**. Настройки пары приёмопередатчиков LoRa должны совпадать (кроме настроек UART).

Настройки LoRa (отмеченные **красным** цветом должны быть одинаковы для пары модулей):

1. **UartRate** - скорость обмена данными по UART (она влияет на настройки при инициализации обмена данными по UART в программе)
2. **Parity** - контроль чётности (настройка UART)
3. **AirRate** - скорость передачи данных “по воздуху”, чем меньше - тем больше дальность связи и меньше потерь данных

4. **Power** - мощность передачи (для тестов в лаборатории 21 dbm, перед полётом перенастроить на 30 dbm)
5. **FEC** - упреждающая коррекция ошибок (должна быть включена, Enabled)
6. **Fixed mode** - позволяет учитывать при передаче адрес устройства, которому адресовано сообщение. Настройка должна быть отключена, Disabled.
7. **WOR timing** - 250 мс, оставить значение по умолчанию (используется только в режиме Power saving mode)
8. **IO drive mode** - оставить значение по умолчанию, 1 (TXD and AUX push-pull outputs, RXD pull-up inputs)
9. **Address** - оставить значение по умолчанию, 0.
10. **Channel** - выбор номера канала (частоты приёмопередачи). Доступные частоты варьируется от 410 до 441 МГц, что соответствует номеру канала от 0 до 31.

При выборе канала не рекомендуется выбирать частоту, широко используемую радиолюбителями - 433 МГц, а также (при возможности, но это не так страшно) частоту 434 МГц, так как она входит в используемый в России диапазон гражданских частот LPD (69 каналов в диапазоне 433,075-434,775 МГц). Также убедитесь перед стратосферным запуском, что выбранная частота для общения с аппаратом не пересекается с частотой, используемой командой организаторов стратосферного запуска. Рекомендуется разносить каналы на 2 МГц.

Конструирование

Форм-фактор конструктора – CubeSat 3U.

Рама спутника собирается из рельсов и перекладин двух типов следующим образом:

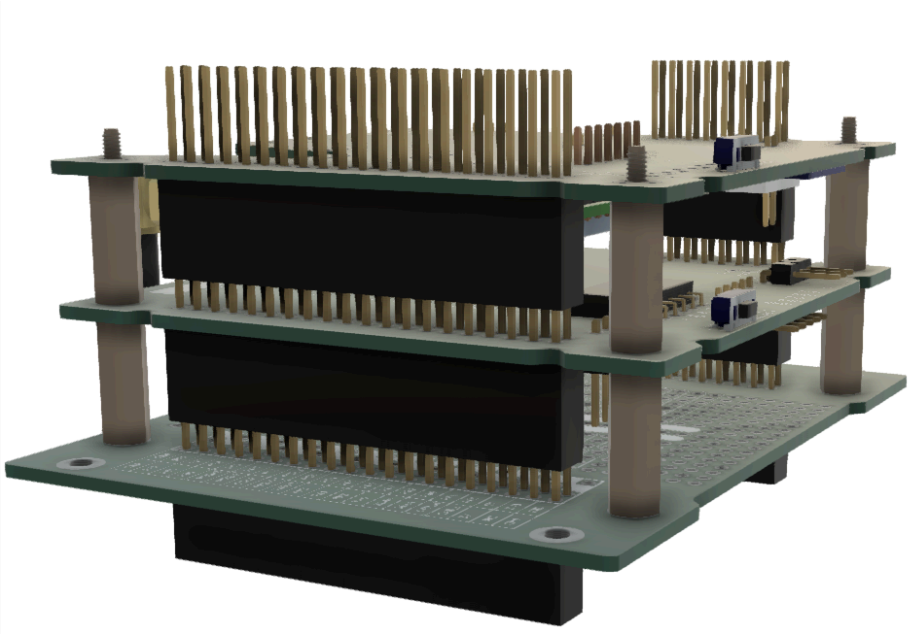
1. Возьмите два рельса и установите перекладины с пазом в выемки, отмеченные на рисунке:



2. В получившиеся углубления на стыке рельса и перекладины установите перекладины с торцевым отверстием так, чтобы поперечные отверстия в этих перекладинах были ориентированы перпендикулярно рельсу (они используются для установки боковых панелей).
3. Закрепите угловое соединение винтами M2.5x12 с потайной головкой.
4. Установите две оставшиеся перекладины и два оставшихся рельса аналогичным образом и окончательно соберите конструкцию при помощи винтов M2.5x12.

Соединённые перекладины двух типов теперь представляют из себя шпангоуты. К верхнему и нижнему шпангоутам крепятся верхняя и нижняя панели с помощью винтов M2.5x6 с потайной головкой. Промежуточный шпангоут служит для крепления платы интерфейсного модуля.

Служебные платы собираются в единый стек вместе с интерфейсным модулем и закрепляются с помощью латунных стоек M2.5 длиной 15 мм.



Платы, соединённые в стек и закреплённые стойками M2.5 длиной 15 мм

Боковые панели и промежуточные шпангоуты изготавливаются учащимися самостоятельно. В одной из боковых панелей необходимо предусмотреть вырезы:

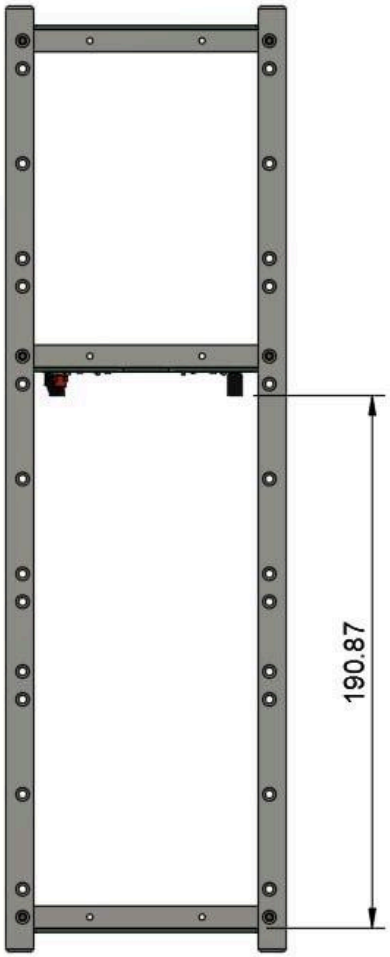
- место для установки ремувки для подачи питания перед полётом
- вырез под крепление антенны
- доступ к кнопкам перезагрузки плат полётного контроллера и связи
- и т.д.

Рекомендуется приступать к проектированию панели с вырезами в последнюю очередь, уточнив у организаторов расположения места для установки ремувки и др.

Полезная нагрузка может быть размещена в объёме, соответствующем CubeSat 2U: верхний юнит при этом занимают служебные системы.



Демонстрация разделения объёма CubeSat 3U на юнит служебных систем (сверху от интерфейсного модуля) и два юнита полезной нагрузки (снизу).



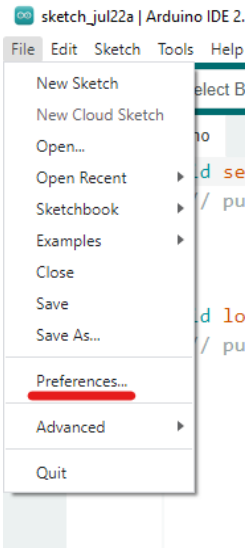
Минимальная высота, на которой может находиться нижний край разъёма платы интерфейсного модуля

Программирование

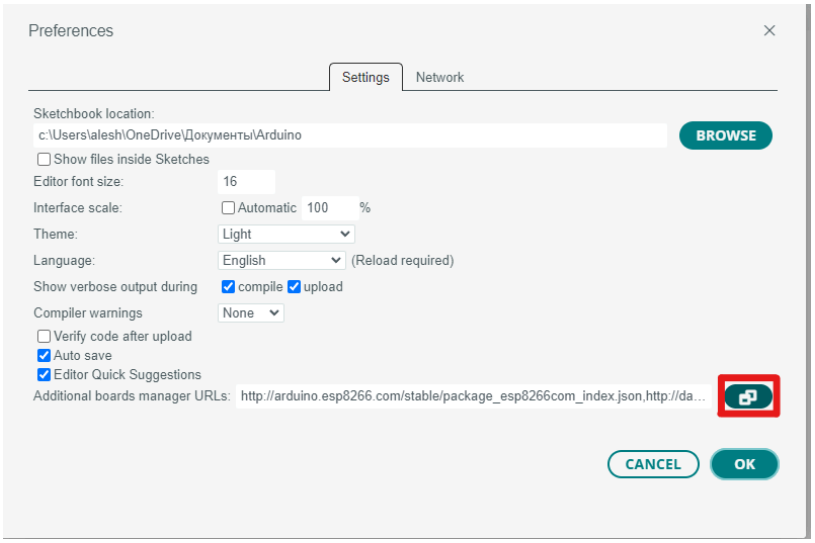
Установка программного обеспечения и библиотек

Для написания программного кода и прошивки полётного контроллера можно использовать ПО Arduino IDE (скачать по ссылке). Для работы с платами семейства STM32 необходимо также установить дополнительный пакет поддержки плат. Для этого в настройках Arduino IDE нужно вставить дополнительную ссылку для менеджера плат:

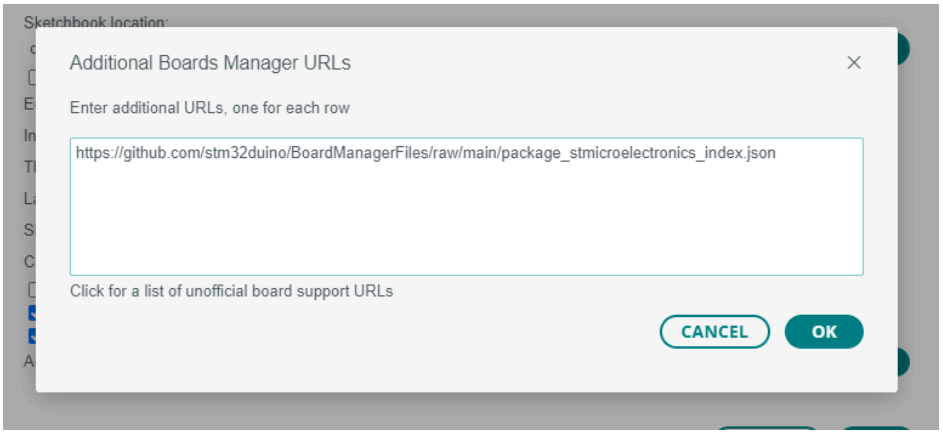
https://github.com/stm32duino/BoardManagerFiles/raw/main/package_stmicroelectronics_index.json



Меню настроек

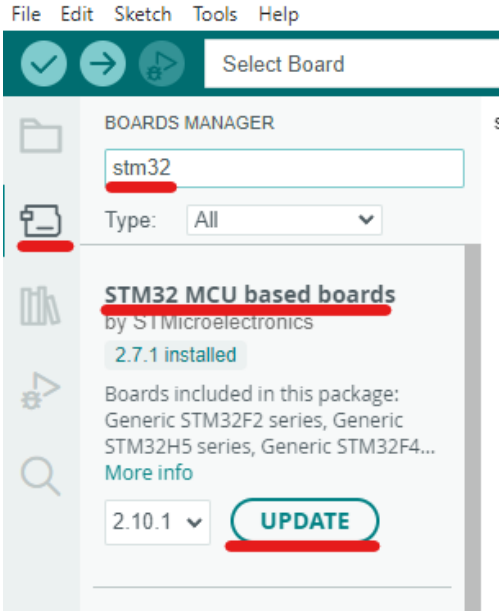


Открытие окна дополнительных ссылок для менеджера плат



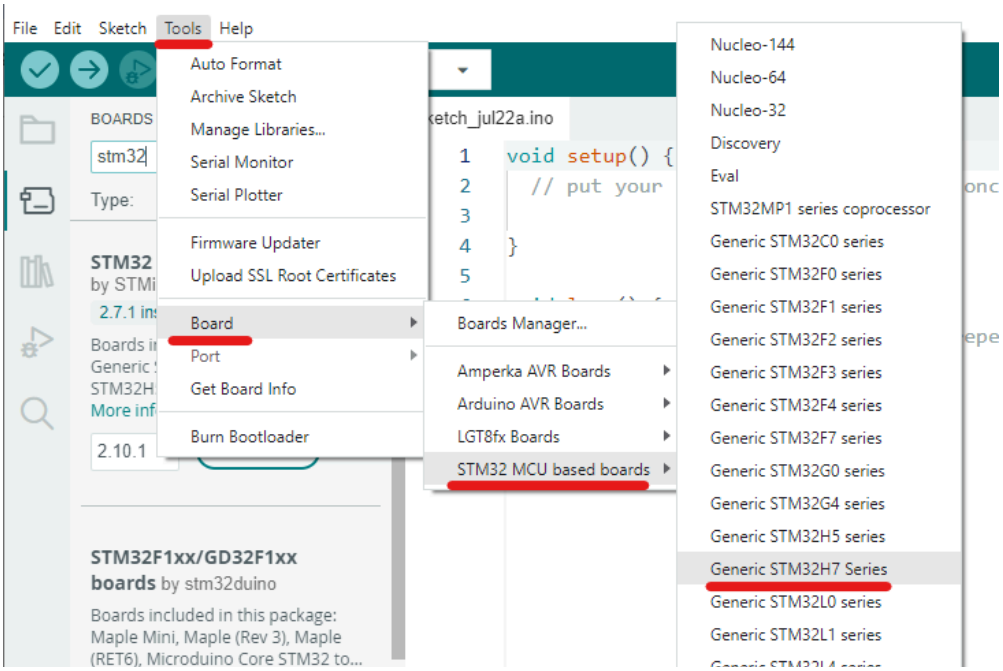
Вставленная ссылка

После необходимо открыть Менеджер плат, ввести в поиске “stm32” и установить пакет “STM32 MCU based boards”. Установка может занять некоторое время в зависимости от интернет-соединения.



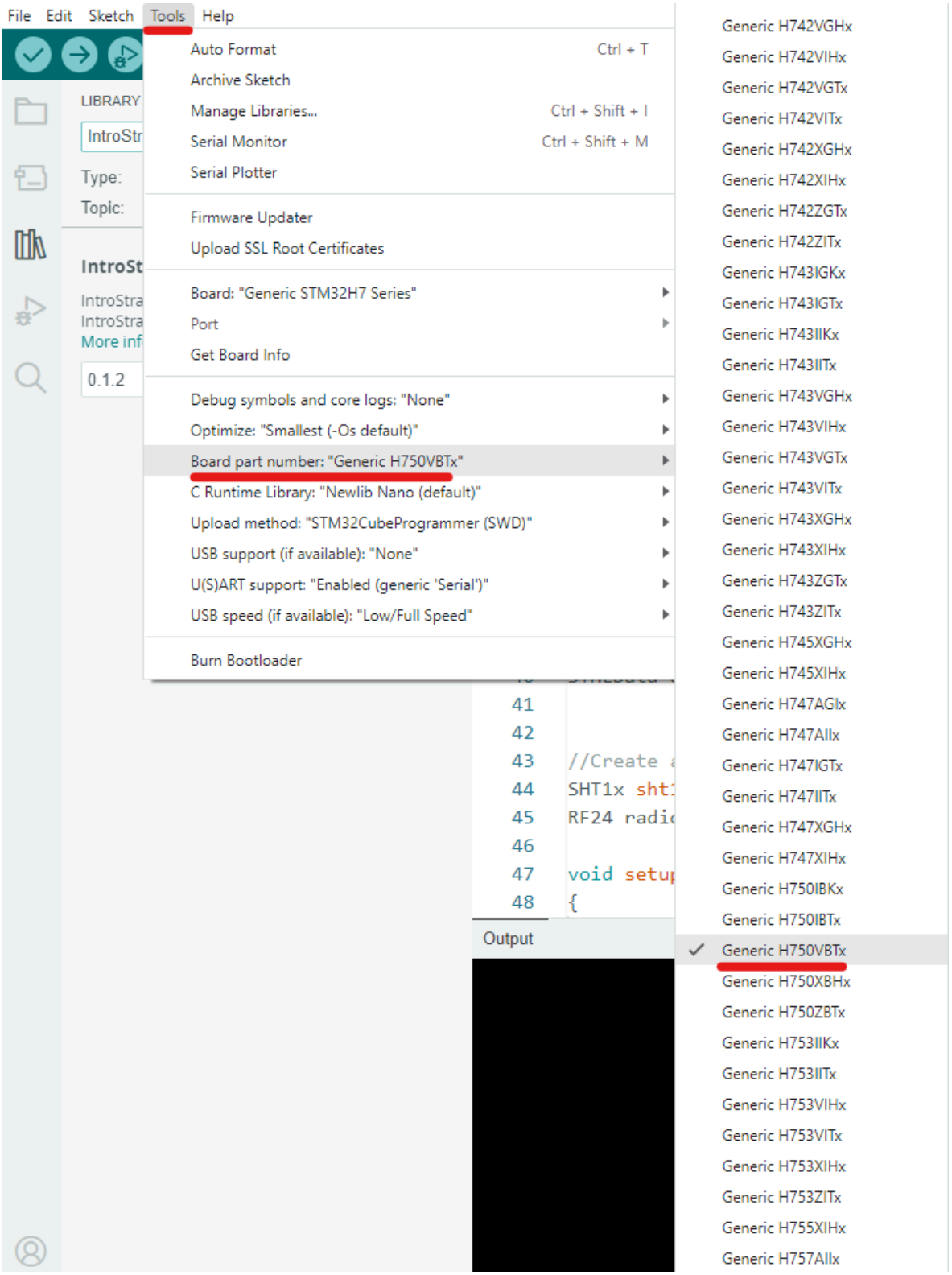
Установка пакета плат

Теперь в меню “Инструменты” можно выбрать соответствующую серию микроконтроллеров.



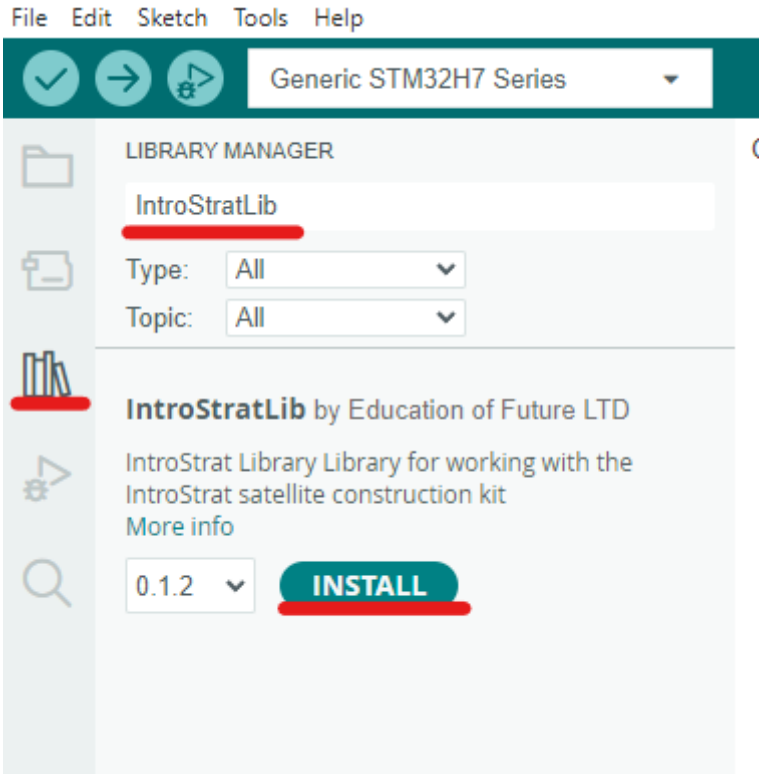
Выбор семейства микроконтроллеров

Далее в том же меню выбираем установленный контроллер (если необходимого микроконтроллера нет в списке, следует пролистать его ниже):



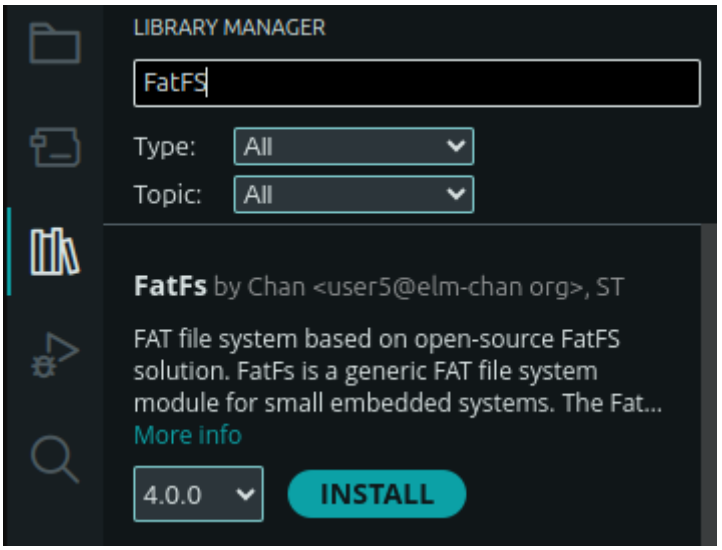
Выбор микроконтроллера

В дальнейшем для удобной работы с платформой рекомендуется использовать библиотеку IntroStratLib. Её можно установить через менеджер библиотек в Arduino IDE.



Установка библиотеки IntroStratLib

Также для сохранения логов и полезных данных на SD карту рекомендуется установить две библиотеки: FatFS и STM32SD. Их также можно найти в менеджере библиотек.



Установка библиотеки FatFS



Установка библиотеки STM32SD

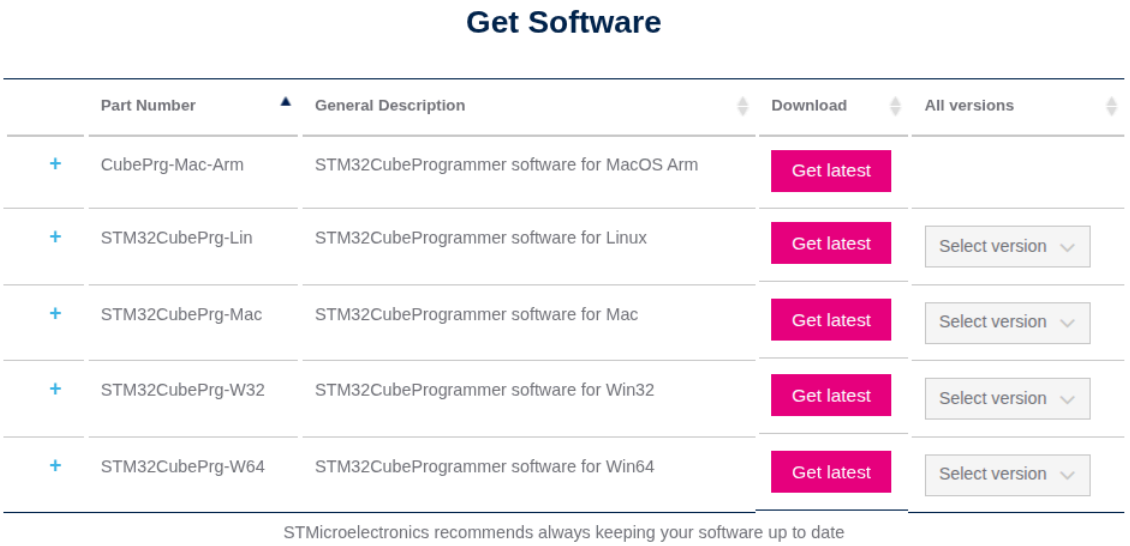
После окончания настройки Arduino IDE также необходимо скачать и установить STM32CubeProgrammer – с ней станет возможна прошивка микроконтроллеров STM32 через Arduino IDE. Ссылка на скачивание (требуется бесплатная регистрация на сайте):

https://www.st.com/en/development-tools/stm32cubeprog.html

If the link above doesn’t work, choose the right OS and download the software from the Google Drive:

https://drive.google.com/drive/folders/16j7ZNJIxbnB-YA9xYJRHTKJxJhtnI8BM?usp=sharing

It may also require installing Java in addition, you need to do it as well.



Скачивание STM32CubeProgrammer

Использование основных интерфейсов

UART

UART (**U**niversal **A**synchronous **R**eceive **T**ransmit) - один из самых распространённых протоколов для передачи данных между устройствами. Он использует две линии данных, по одной устройство передаёт данные, а по другой принимает. На стороне каждого устройства контакты, к которым подключены эти линии, называются **RX** (Receive - для приёма данных) и **TX** (Transmit - для передачи). На плате FCM на разъём IntroBus-L выведено 3 интерфейса UART: **UART3**, **UART5**, **UART8**.

Ещё один интерфейс (**UART1**) выведен на разъём для подключения Bluetooth-модуля, и на разъём IntroBus-S, для подключения платы связи.

Начало работы с UART

Для работы с любым из этих интерфейсов необходимо создать объект класса `HardwareSerial` с указанием пинов, на которых создаётся интерфейс. В общем виде создание объекта выглядит следующим образом:

```
HardwareSerial MySerial(RX_pin, TX_pin);
```

Для интерфейсов, используемых на плате полётного контроллера, могут потребоваться следующие объекты:

```
HardwareSerial Serial1(PA10, PA9);
HardwareSerial Serial3(PD9, PD8);
```

```
HardwareSerial Serial5(PB12, PB13);
HardwareSerial Serial8(PE0, PE1);
```

После этого с каждым из них можно работать так же, как с привычном объектом Serial в Arduino. Вместо `Serial1` можно использовать любое другое название.

Инициализация

В функции `setup()` необходимо задать настройки UART:

```
Serial1.begin(115200, SERIAL_8E1);
```

Эта строчка инициализирует заданный ранее **UART1** на скорости **115200** бод, с использованием **8** бит данных, проверкой на чётность **Even** и **одним** стоп-битом.

Передача данных

Для передачи данных можно использовать стандартные методы Serial.

```
Serial1.print("Hello world!"); // передача строки
Serial1.write(0x14); // передача байта
```

Приём данных

STM32duino реализует буферизацию входящих данных, что позволяет использовать стандартный метод `available()` для проверки наличия пришедших данных.

```
if (Serial1.available() > 0) {
    uint8_t data = Serial1.read(); // читаем 1 байт из
пришедших данных
    // делаем что-то с data
}
```

I2C

I2C (**I**nter-**I**ntegrated **C**ircuit) - распространённый протокол для соединения периферийных устройств в общую шину. Он использует две линии:

SDA - для получения и передачи данных

SCL - для тактирования

На шине всегда должно быть одно *ведущее (master)* устройство, и до 127 *ведомых (slave)*. Любой обмен данными инициализируется ведущим устройством, будь то запрос данных с датчика или запись значения в периферийное устройства. В нашел случае ведущим устройством выступает микроконтроллер на плате FCM. С него выведено два интерфейса I2C:

I2C1 - системная шина. К ней подключены все датчики и устройства, установленные на платформе.

I2C4 - шина для подключения устройств полезной нагрузки.

Начало работы с I2C

Для работы с любым из этих интерфейсов необходимо создать объект класса `TwoWire` с указанием пинов, на которых создаётся интерфейс. В общем виде это выглядит следующим образом:

```
TwoWire MyWire(PB7, PB6);
```

Для системной шины и шины для подключения полезной нагрузки потребуются следующие объекты:

```
TwoWire Wire1(PB7, PB6);
TwoWire Wire4(PD13, PD12);
```

После этого с каждым из них можно работать так же, как с привычным объектом Wire в Arduino. Вместо `Wire1` можно использовать любое другое название.

Инициализация

В функции `setup()` необходимо инициализировать I2C:

```
Wire1.begin();
```

Вызов метода `begin()` без параметров инициализирует интерфейс в режиме ведущего устройства и позволяет работать с периферийными устройствами.

Обмен данными

Для общения с периферийными устройствами (например, датчиками), рекомендуется использовать объекты и методы из библиотеки IntroStratLib. В следующей главе подробно рассмотрен процесс обмена данными с датчиками.

Если в полезной нагрузке используются устройства, работающие по I2C, есть два способа организовать обмен данными.

Попробуйте найти готовую библиотеку для работы с используемым устройством. Такие библиотеки обычно работают с объектом `TwoWire`, который мы создали и инициализировали ранее.

Если готовой библиотеки нет, или вы хотите сами написать логику взаимодействия, можете подробнее изучить справку Arduino по работе с TwoWire: https://docs.arduino.cc/learn/communication/wire/

SPI

SPI (**S**erial **P**eripheral **I**nterface) – ещё один распространённый протокол обмена данными между микроконтроллером, выступающим в качестве *ведущего* устройства, и периферией (*ведомыми* устройствами). Для данного протокола нужно четыре линии:

SS – Slave Select, выбор ведомого

MOSI – Master Out Slave In, данные передаются от ведущего к ведомому

MISO – Master In Slave Out, ведущий принимает данные от ведомого

SCK – линия тактирования, по которой передаются импульсы синхронизации

Для каждого ведомого устройства нужен отдельный контакт SS. Ведущее устройство прижимает линию SS к земле, чтобы инициировать обмен данными с соответствующим ведомым.

На плате FCM выведен интерфейс SPI2 (сигналы MOSI2, MISO2 и SCK2). Любой свободный контакт шины IntroBus_L можно использовать в качестве линии SS, подключив к нему контакт SS ведомого устройства.

Начало работы с SPI

Для работы с SPI требуется создать объект класса `SPIClass` с указанием пинов, на которых создаётся интерфейс. В общем виде это выглядит следующим образом:

```
SPIClass MySPI(MOSIpin, MISOpin, SCLKpin);
```

Для интерфейса SPI2 платы FCM требуется создать объект следующим образом:

```
SPIClass SPI_2(PB15, PB14, PB10);
```

Также для удобства можно инициализировать константу, содержащую в себе номер пина, к которому подключена линия SS ведомого устройства, например:

```
const int slaveSelect = PB1;
```

Инициализация

В функции `setup()` необходимо инициализировать SPI:

```
SPI_2.begin();
```

Также необходимо обозначить, что вывод `slaveSelect`, объявленный константой ранее, будет использоваться в качестве выхода:

```
pinMode(slaveSelect, OUTPUT);
```

Обмен данными

Если в полезной нагрузке используются устройства, работающие по SPI, есть два способа организовать обмен данными.

Попробуйте найти готовую библиотеку для работы с используемым устройством. Если библиотека не рассчитана на работу с конкретным объектом класса `SPIClass`, можно попробовать вручную переназначить пины стандартного SPI в функции `setup()`:

```
SPI.setMOSI(PB15);

SPI.setMISO(PB14);

SPI.setSCLK(PB10);

SPI.setSSEL(PB1);
```

Если готовой библиотеки нет, или вы хотите сами написать логику взаимодействия, можете подробнее изучить справку Arduino по работе с SPI: https://docs.arduino.cc/language-reference/en/functions/communication/SPI/SPISettings/

Работа со служебными датчиками

Общая информация
Подключение всех служебных датчиков, расположенных на плате полётного контроллера, осуществляется через шину I2C1. Подробнее о работе по интерфейсу I2C можно прочитать выше.

Работать с датчиками рекомендуется с использованием библиотеки IntroStratLib (здесь описано, как её установить).

На плате FCM установлено четыре служебных датчика:

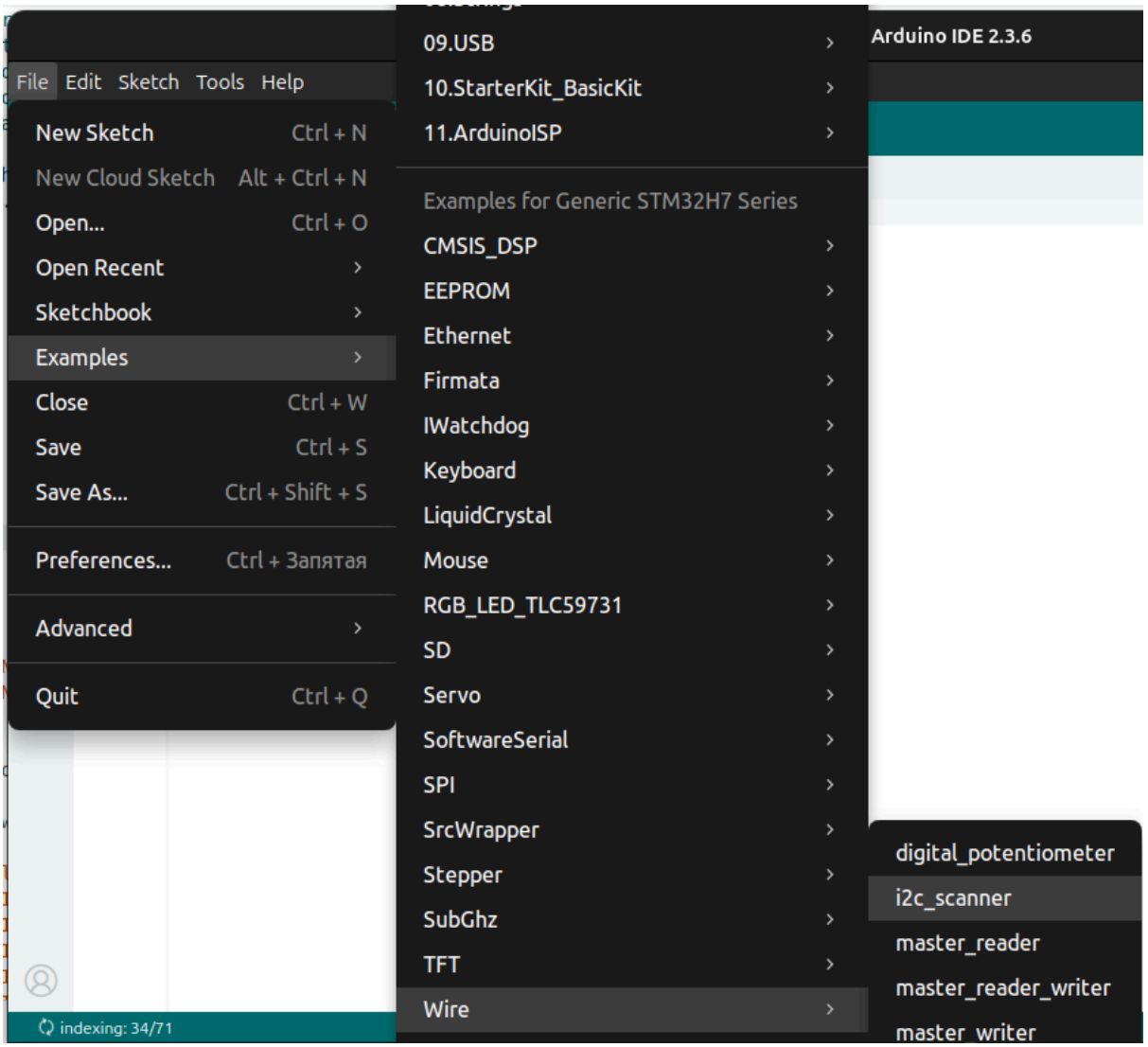
Название датчика	**Измеряемые величины**	**Используемая библиотека**	**Адрес на шине I2C**
LSM6DS3 (гироскоп и акселерометр)	угловая скорость (град/с), линейное ускорение (g)	`LSM6DS3.h`	0x6A
LIS3MDL (магнитометр)	магнитная индукция (Гаусс)	`LIS3MDL.h`	0x1C
MS5611 (барометр)	давление (мбар), температура (°C)	`MS5611.h`	0x77
LM75A (датчик температуры)	температура (°C)	`LM75A.h`	0x4A

I2C Scanner
I2C Scanner – пример из стандартной библиотеки Arduino `Wire.h`, позволяющий установить адреса устройств, исправно подключенных к шине I2C. Микроконтроллер перебирает все 127 семибитных адреса и пытается установить связь с устройством, предположительно находящимся на каждом адресе. Если связь установлена успешно, в монитор порта выводится надпись, что к шине подключено устройство с данным адресом. Этот скетч полезно использовать в некоторых случаях:

- при ознакомлении с платой FCM, чтобы увидеть, на каких адресах находятся служебные датчики

- для проверки правильного использования шины I2C в коде, чтобы убедиться, что объект интерфейса создан верно и используются правильные пины
- если с датчика приходят неверные данные, для первичной отладки можно проверить, отвечает ли он на запросы ведущего устройства

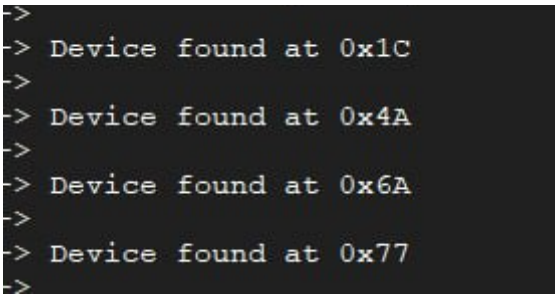
Скетч I2C Scanner можно найти в примерах библиотеки следующим образом: в Arduino IDE нажмите *File → Examples → Wire → i2c_scanner*.



Для работы с платой FCM в скетч нужно внести некоторые изменения:

- создайте объект интерфейса I2C, который вы хотите просканировать
- создайте объект интерфейса UART, в который вы хотите выводить данные, чтобы отслеживать результат в мониторе порта
- замените используемые в скетче объекты Serial и Wire на созданные вами

Если вы сканируете служебную шину I2C, то в результате вы должны увидеть как минимум четыре адреса служебных датчиков платы FCM.

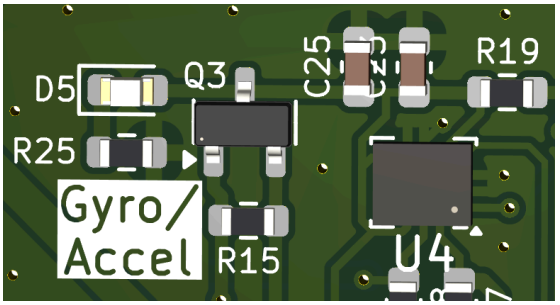


Пример сканирования служебной шины I2C1

Гироскоп

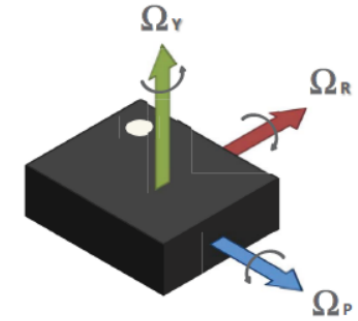
Датчик LSM6DS3 позволяет измерять угловые скорости по трём осям в нескольких диапазонах с различной точностью (чем меньше диапазон, тем больше точность, так как одно измеренное значение всегда занимает 2 байта в соответствующих регистрах датчика).

Датчик LSM6DS3 обозначен на плате FCM как **U4**. Он расположен следующим образом:



U4 – LSM6DS3

Согласно спецификации на датчик, оси, вокруг которых происходит измерение угловых скоростей, расположены следующим образом:



Direction of detectable
angular rate (top view)

По ключу (точке) на корпуса датчика можно определить направления осей измерения на плате FCM.

Подробную информацию о датчике можно получить по ссылке:

https://www.st.com/resource/en/application_note/an5130-lsm6ds3trc-alwayson-3d-accelerometer-and-3d-gyroscope-stmicroelectronics.pdf

Для получения данных с гироскопа в скетче в первую очередь необходимо подключить заголовочный файл:

```
#include <LSM6DS3.h>
```

Затем нужно включить пространство имён библиотеки IntroStratLib:

```
using namespace IntroStratLib;
```

Создайте объект класса `LSM6DS3` для работы с датчиком:

```
LSM6DS3 sm(MYWIRE1, 0x6A);
```

`MYWIRE1` – объект служебной шины I2C, `0x6A` – адрес датчика. Название `sm` может быть любым.

В функции `setup()` инициализируйте служебную шину I2C и датчик. Инициализация датчика в данном случае задаёт настройки гироскопа. Рекомендуется после каждой инициализации добавлять небольшую задержку, например, 100 мс:

```
MYWIRE1.begin();
delay(100);
sm.InitGyro();
delay(100);
```

Инициализация происходит с настройками по умолчанию:

- data rate 104 Гц
- диапазон измерений ±500 dps

Можно инициализировать датчик с желаемыми настройками, указав их в качестве аргументов метода `InitGyro()`. Подробную информацию о настройках можно найти в исходном коде библиотеки `LSM6DS3`.

Для получения “сырых” данных угловой скорости (значение угловой скорости по одной оси занимает два байта) существуют методы RawGX, RawGY и RawGZ, возвращающие значение типа данных *int16_t*:

```
sm.RawGX();
sm.RawGY();
sm.RawGZ();
```

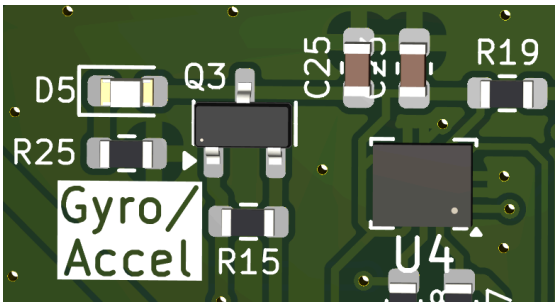
Для получения показаний угловой скорости в градусах в секунду существуют методы GX, GY и GZ, возвращающие значение типа данных *float*:

```
sm.GX();
sm.GY();
sm.GZ();
```

Акселерометр

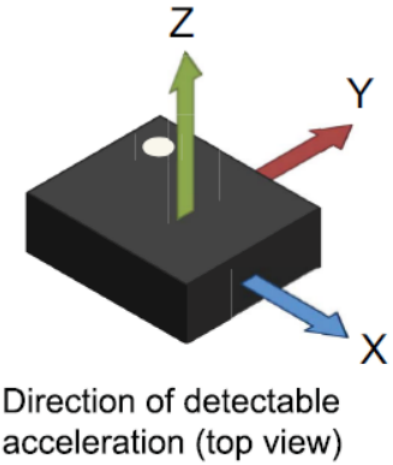
Датчик LSM6DS3 также позволяет измерять линейные ускорения по трём осям в нескольких диапазонах с различной точностью (чем меньше диапазон, тем больше точность, так как одно измеренное значение всегда занимает 2 байта в соответствующих регистрах датчика).

Датчик LSM6DS3 обозначен на плате FCM как **U4**. Он расположен следующим образом:



U4 – LSM6DS3

Согласно спецификации на датчик, оси, в направлении которых происходит измерение линейных ускорений, расположены следующим образом:



Direction of detectable acceleration (top view)

По ключу (точке) на корпуса датчика можно определить направления осей измерения на плате FCM.

Подробную информацию о датчике можно получить по ссылке:

https://www.st.com/resource/en/application_note/an5130-lsm6ds3trc-alwayson-3d-accelerometer-and-3d-gyroscope-stmicroelectronics.pdf

Для получения данных с акселерометра в скетче в первую очередь необходимо подключить заголовочный файл:

```
#include <LSM6DS3.h>
```

Затем нужно включить пространство имён библиотеки IntroStratLib:

```
using namespace IntroStratLib;
```

Создайте объект класса `LSM6DS3` для работы с датчиком:

```
LSM6DS3 sm(MYWIRE1, 0x6A);
```

`MYWIRE1` – объект служебной шины I2C, `0x6A` – адрес датчика. Название `sm` может быть любым.

В функции `setup()` инициализируйте служебную шину I2C и датчик. Инициализация датчика в данном случае задаёт настройки акселерометра. Рекомендуется после каждой инициализации добавлять небольшую задержку, например, 100 мс:

```
MYWIRE1.begin();
delay(100);
sm.InitAccel();
delay(100);
```

Инициализация происходит с настройками по умолчанию:

- data rate 104 Гц
- диапазон измерений ±4g

Можно инициализировать датчик с желаемыми настройками, указав их в качестве аргументов метода `InitAccel()`. Подробную информацию о настройках можно найти в исходном коде библиотеки `LSM6DS3`.

Для получения “сырых” данных линейного ускорения (значение линейного ускорения по одной оси занимает два байта) существуют методы RawAX, RawAY и RawAZ, возвращающие значение типа данных *int16_t*:

```
sm.RawAX();
sm.RawAY();
sm.RawAZ();
```

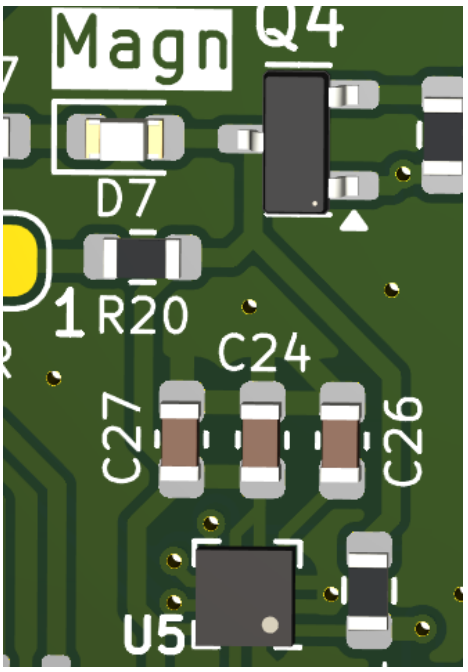
Для получения показаний линейного ускорения в долях от ускорения свободного падения **g** существуют методы AX, AY и AZ, возвращающие значение типа данных *float*:

```
sm.AX();
sm.AY();
sm.AZ();
```

Магнитометр

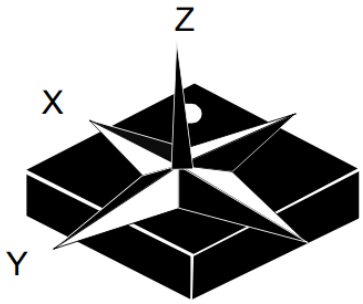
Датчик LIS3MDL позволяет измерять магнитную индукцию по трём осям в нескольких диапазонах с различной точностью (чем меньше диапазон, тем больше точность, так как одно измеренное значение всегда занимает 2 байта в соответствующих регистрах датчика).

Датчик LIS3MDL обозначен на плате FCM как **U5**. Он расположен следующим образом:



U5 – LIS3MDL

Согласно спецификации на датчик, оси, в направлении которых происходит измерение магнитной индукции, расположены следующим образом:



TOP VIEW

DIRECTION OF
DETECTABLE
MAGNETIC FIELDS

По ключу (точке) на корпуса датчика можно определить направления осей измерения на плате FCM.

Подробную информацию о датчике можно получить по ссылке:

https://www.st.com/resource/en/datasheet/lis3mdl.pdf

Для получения данных с магнитометра в скетче в первую очередь необходимо подключить заголовочный файл:

```
#include <LIS3MDL.h>
```

Затем нужно включить пространство имён библиотеки IntroStratLib:

```
using namespace IntroStratLib;
```

Создайте объект класса `LIS3MDL` для работы с датчиком:

```
LIS3MDL magn(MYWIRE1, 0x1C);
```

`MYWIRE1` – объект служебной шины I2C, `0x1C`– адрес датчика. Название `magn` может быть любым.

В функции `setup()` инициализируйте служебную шину I2C и датчик. Рекомендуется после каждой инициализации добавлять небольшую задержку, например, 100 мс:

```
MYWIRE1.begin();
delay(100);
magn.Init();
delay(100);
```

Инициализация происходит с настройками по умолчанию:

- диапазон измерений ±16 Гаусс

Можно инициализировать датчик с желаемым диапазоном измерений, указав его в качестве аргумента метода `Init()`. Подробную информацию о настройках можно найти в исходном коде библиотеки `LIS3MDL`.

Для получения “сырых” данных магнитной индукции (значение магнитной индукции по одной оси занимает два байта) существуют методы RawMX, RawMY и RawMZ, возвращающие значение типа данных *int16_t*:

```
magn.RawMX();
magn.RawMY();
magn.RawMZ();
```

Для получения показаний магнитной индукции в Гауссах существуют методы MX, MY и MZ, возвращающие значение типа данных *float*:

```
magn.MX();
magn.MY();
magn.MZ();
```

Барометр

Датчик MS5611 позволяет измерять температуру и давление в диапазоне от 10 до 1200 мбар, благодаря чему можно вычислить высоту даже при запуске в стратосферу, в условиях очень разреженной атмосферы.

Подробную информацию о датчике можно получить по ссылке:

https://www.te.com/commerce/DocumentDelivery/DDEController?Action=showdoc&DocId=Data+Sheet%7FMS5611-01BA03%7FB3%7Fpdf%7FEnglish%7FENG_DS_MS5611-01BA03_B3.pdf%7FCAT-BLPS0036

Для получения данных с барометра в скетче в первую очередь необходимо подключить заголовочный файл:

```
#include <MS5611.h>
```

Затем нужно включить пространство имён библиотеки IntroStratLib:

```
using namespace IntroStratLib;
```

Создайте объект класса `MS5611` для работы с датчиком:

```
MS5611 bar(MYWIRE1, 0x77);
```

`MYWIRE1` – объект служебной шины I2C, `0x77`– адрес датчика. Название `bar` может быть любым.

В функции `setup()` инициализируйте служебную шину I2C и датчик. Рекомендуется после каждой инициализации добавлять небольшую задержку, например, 100 мс:

```
MYWIRE1.begin();
delay(100);
bar.Init();
delay(100);
```

Инициализация происходит с настройками по умолчанию:

- oversampling rate (OSR) – 1024, что соотвутетсвует погрешности измерений датчика 0.027 мбар и 0.005 °C.

Можно инициализировать датчик с другой желаемой точностью измерений, указав другое значение OSR в качестве аргумента метода `Init()`. Подробную информацию о настройках можно найти в исходном коде библиотеки `MS5611`.

Для получения значения температуры в градусах Цельсия используется метод GetTemperature(), возвращающий значение типа данных *float*:

```
bar.GetTemperature();
```

Для получения значения давления в миллибарах используется метод GetPressure(), возвращающий значение типа данных *float*:

```
bar.GetPressure();
```

Оценить приблизительное значение высоты в метрах можно с помощью метода GetHeight(), возвращающего значение типа данных *float*:

```
bar.GetHeight();
```

Существуют различные формулы, позволяющие вычислить высоту через давление и температуру, и вы можете самостоятельно произвести вычисления с помощью любой из них. Наибольшей точности можно добиться, если учитывать перед измерением меняющейся высоты начальные условие, такие как высоту над уровнем местности, атмосферное давление, температуру воздуха у поверхности Земли. Подробную информацию о точных вычислениях можно прочитать по ссылке:

https://www.amsys-sensor.com/downloads/notes/ms5611-precise-altitude-measurement-with-a-pressure-sensor-module-amsys-509e.pdf

Датчик температуры

Датчик LM75AD позволяет измерять температуру в диапазоне от −55 °C до +125 °C с разрешением в 0.125 °C. Этот датчик имеет настраиваемый адрес, что позволяет подключать несколько датчиков к одной шине I2C. На платформе такие датчики установлены на плате полётного контроллера (адрес 0x4A) и на плате связи (адрес 0x4C).

Подробную информацию о датчике можно получить по ссылке:

https://www.nxp.com/docs/en/data-sheet/LM75A.pdf

Для получения данных с датчика температуры в скетче в первую очередь необходимо подключить заголовочный файл:

```
#include <LM75A.h>
```

Затем нужно включить пространство имён библиотеки IntroStratLib:

```
using namespace IntroStratLib;
```

Создайте объект класса `LM75A` для работы с датчиком:

```
LM75A temp(MYWIRE1, 0x4A);
```

`MYWIRE1` – объект служебной шины I2C, `0x4A`– адрес датчика. Название `temp` может быть любым.

В функции `setup()` инициализируйте служебную шину I2C и датчик. Рекомендуется после каждой инициализации добавлять небольшую задержку, например, 100 мс:

```
MYWIRE1.begin();
delay(100);
temp.Init();
delay(100);
```

Для получения значения температуры в градусах Цельсия используется метод `GetTemperature()`, возвращающий значение типа данных *float*:

```
temp.GetTemperature();
```

Так как значения температура с датчика достаточно дискретные (резрешение измерения в 0.125 °C), то если умножить значение температуры на 8, его можно поместить в 2 байта (переменную типа int16_t) без потери точности. Это позволит сэкономить место (размер переменной типа *float* - 4 байта), например при отправке данных телеметрии. Для получения температуры в таком виде можно использовать метод `GetTemperatureTimes8()` , возвращающий целое знаковое число - значение температуры умноженное на 8.

```
temp.GetTemperatureTimes8();
```

Сохранение данных на microSD карту

На материнской плате расположен слот для подключения microSD-карты, на которой можно сохранять данные. Это может пригодиться для логирования информации во время проведения экспериментов, когда макет спутника работает автономно, чтобы полезные данные не потерялись из-за возможных перебоев со связью по радиоканалу.

Подключение microSD карты к микроконтроллеру на плате FCM происходит по интерфейсу SDMMC – специальному интерфейсу для обмену данными с картами памяти различных типов, а конкретно – SDMMC2.

Для записи и чтения информации на microSD карту рекомендуется использовать библиотеку `STM32SD`, которая в свою очередь базируется на библиотеке `FatFS`. Достаточно подключить заголовочный файл библиотеки `STM32SD`:

```
#include <STM32SD.h>
```

Так как слот SD карты позволяет определить, вставлена ли в данный момент карта, нужно определить константу с названием `SD_DETECT_PIN` и указать пин микроконтроллера, куда подключен контакт детектирования карты. В нашем случае это пин PB8.

```
#ifndef SD_DETECT_PIN
#define SD_DETECT_PIN PB8
#endif
```

Если константа с названием `SD_DETECT_PIN` не определена в коде, то при работе с библиотекой просто не происходит проверка детектирования карты.

Также необходимо создать объект класса `File` библиотеки `STM32SD`: он понадобится для дальнейшей работы с microSD картой:

```
File dataFile;
```

В функции `setup()` необходимо перенастроить пины стандартного интерфейса SDMMC, чтобы они совпадали с пинами, используемыми на плате:

```
SD.setDx(PC8, PC9, PC10, PC11);
SD.setCMD(PD2);
SD.setCK(PC12);
```

Далее необходимо инициализировать обмен данными с картой памяти. Сделаем это в цикле, проверяя в условии возвращаемое методом `begin()` значение:

```
while (!SD.begin());
```

При этом, если карта не вставлена в слот, метод `begin()` вернёт ненулевое значение; таким образом код не будет выполняться дальше при отсутствии карты памяти. После соединения карты памяти с платой FCM необходимо перезагрузить плату.

Теперь требуется открыть файл и сохранить значение, возвращаемое методом `open`, в созданный ранее объект `dataFile` класса `File`:

```
dataFile = SD.open("datalog.txt", FILE_WRITE);
```

Аргументы метода :

- `"datalog.txt"` – название файла, может быть любым
- `FILE_WRITE` – атрибут открытия файла. С этим атрибутом файл открывается для записи; запись происходит в конец файла. Также можно использовать атрибут `FILE_READ` – открытие файла только для чтения.

Нужно проверить, успешно ли открыт файл – только в этом случае можно осуществлять попытки записи и чтения в него:

```
if (!dataFile) {
   // if the file isn't open, pop up an error
}
```

Используйте это условие для отладки – в тело условия можно прописать вывод сообщения об ошибке в монитор порта.

Запишем данные формата *String* в успешно открытый файл с помощью метода `println()`. После чего необходимо вызвать метод `flush()`, чтобы убедиться, что данные будут записаны полностью:

```
String dataString = "hello";
if (dataFile) {
  dataFile.println(dataString);
  dataFile.flush();
}
```

Таким образом также можно записывать данные других форматов данных.

Более подробную информацию о методах работы с файлами можно, изучив библиотеку `STM32SD`; можно пользоваться примерами из неё.